chiselWare

Developers Guide

For Chisel-based Semiconductor IP Cores

Version 0.9.0

November 4, 2026

https://chiselware.org

Copyright © 2026 Rocksavage Technology, Inc. All Rights Reserved

Notice

This document is a working draft of the chiselWare Standard (CWS). It is published for review and comment purposes only. The contents of this document are subject to change without notice.

Copyright © 2026 Rocksavage Technology, Inc. All rights reserved.

No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of Rocksavage Technology, Inc., except for the purpose of review and comment during the standards development process.

chiselWare, the chiselWare logo, and IP Factory are trademarks of Rocksavage Technology, Inc. All other trademarks are the property of their respective owners.

Inquires can be made to:

Rocksavage Technology, Inc.
1821 S. Bascom Avenue, Suite 374
Campbell, CA 95008

Email: admin@chiselware.org

Abstract

The chiselWare® Developers Guide (CWDG) is the practical companion to the chiselWare Standard® (CWS). Where the CWS defines what is required for chiselWare Certification, this guide explains how to meet those requirements efficiently and with confidence. It is organized as a tutorial that follows the natural workflow of chiselWare Core development — from setting up the Standard Software Environment and cloning the template repository, through designing, verifying, and documenting a core, to submitting for certification and maintaining a certified core over time.

This guide is addressed to Core Developers: engineers who want to build professional-grade semiconductor IP using Chisel and contribute it to the chiselWare ecosystem. No prior familiarity with chiselWare is assumed, though working knowledge of Scala, Chisel, and Git is expected. Readers who are new to Chisel will find pointers to learning resources in Appendix D. Readers who are already familiar with the chiselWare infrastructure will find the appendices and CWS cross-references useful as a day-to-day reference.

Keywords: chiselWare, Chisel, semiconductor IP, hardware description language, IP reuse, certification, Scala, FPGA, ASIC, Automated Design Review.

Version	Date	Description
0.9.0	November 4, 2026	Update for new templates
0.8.4	September 26, 2026	Tweaks to docker use model
0.8.3	July 23, 2026	Tweaks to CWDG regarding customization
0.8.2	July 20, 2026	Dev Containers and authorship policy
0.8.1	June 7, 2026	Added citation information for academics
0.8.0	June 4, 2026	Add formal verification support
0.7.5	May 17, 2026	Reworked remove redundancy with Standard
0.7.4	May 7, 2026	No changes; align version with Standard
0.7.2	April 11, 2026	Add Docker
0.6.0	February 21, 2026	Add IPF interfaces
0.5.0	January 27, 2026	Added scalafix support
0.4.0	December 23, 2025	Fixed bug in RunScriptFile
0.3.0	December 3, 2025	Tweaks to Azure Terraform code
0.2.0	November 23, 2025	Update per new Dff template repo
0.1.0	September 18, 2025	Working Draft

Contents

1 Introduction **4**
- 1.1 Why Chisel? . . . 4
- 1.2 Why chiselWare? . . . 5
- 1.3 The Three Tenets . . . 5
  - 1.3.1 Be Kind to the Core User . . . 6
  - 1.3.2 Be Kind to the Core Developer . . . 6
  - 1.3.3 Consistency through Automation . . . 6
- 1.4 The chiselWare Ecosystem . . . 6
  - 1.4.1 The chiselWare Standard (CWS) . . . 7
  - 1.4.2 The chiselWare Core Registry . . . 7
  - 1.4.3 The Standard Software Environment (SSE) . . . 7
  - 1.4.4 The Automated Design Review (ADR) . . . 8
  - 1.4.5 IP Factory . . . 8
  - 1.4.6 The 00-000-dff Template Repository . . . 10
- 1.5 How to Use This Guide . . . 10

2 Before You Begin **12**
- 2.1 What You Should Already Know . . . 12
  - 2.1.1 Scala . . . 12
  - 2.1.2 Chisel . . . 12
  - 2.1.3 Git and GitHub . . . 13
  - 2.1.4 LaTeX . . . 13
- 2.2 Setting Up the Standard Software Environment . . . 13
  - 2.2.1 Docker Image (Recommended) . . . 13
  - 2.2.2 Azure Virtual Machine . . . 14
  - 2.2.3 Self-Built Environment . . . 14
  - 2.2.4 A Note on VS Code . . . 15
- 2.3 Registering with chiselWare . . . 15
  - 2.3.1 Step 1 — Authenticate with LinkedIn . . . 15
  - 2.3.2 Step 2 — Complete Your Developer Profile . . . 15
  - 2.3.3 Step 3 — Execute the Developer Agreement . . . 16
  - 2.3.4 Step 4 — Register or Join an Organization . . . 16
  - 2.3.5 Step 5 — Register or Join a Team . . . 16
  - 2.3.6 Step 6 — Request a Core Project . . . 16

3 Getting Started with the Template **18**

3.1 The 00-000-dff Template Core . . . 18
3.2 Cloning Your Repository . . . 18
3.3 Tour of the Repository . . . 20
3.4 The Customizations . . . 21
3.4.1 Customization 1 — Rename the Directories . . . 21
3.4.2 Customization 2 — Update the Makefile . . . 22
3.4.3 Customization 3 — Update build.sbt . . . 22
3.4.4 Customization 4 — Update the GitHub Actions Workflow . . . 23
3.4.5 Customization 5 — Update README.md, LICENSE.md, CHANGELOG.md 23
3.4.6 Customization 6 — Rename and Update the Scala Files . . . 23
3.4.7 Customization 7 — Configure the Parameter Companion Object . . . 24
3.4.8 Customization 8 — Update the Simulation Testbench . . . 26
3.4.9 Customization 9 — Update the Test Suite . . . 26
3.4.10 Customization 10 — Formal Verification (Optional) . . . 27
3.5 Running make all for the First Time . . . 27
3.6 The Branch Workflow . . . 27
3.6.1 The Three Branches . . . 28
3.6.2 The Development Workflow . . . 28
3.7 Committing Your Starting Point . . . 29

4 Designing Your Core 30
4.1 The Parameter Case Class . . . 30
4.1.1 Philosophy . . . 30
4.1.2 Structure . . . 31
4.1.3 Require Statements . . . 31
4.1.4 Internal Parameterization . . . 31
4.2 The Parameter Companion Object . . . 32
4.2.1 MainClassName . . . 32
4.2.2 simConfigMap . . . 32
4.2.3 synConfigMap . . . 33
4.2.4 fromMap . . . 34
4.3 The SDC File Generator . . . 34
4.3.1 Setting the Target Frequency . . . 35
4.3.2 Port Names in SDC . . . 35
4.4 The IP Factory Parameters Map . . . 35
4.4.1 FieldType . . . 35
4.4.2 pickList . . . 36
4.4.3 message . . . 36
4.5 The Top-Level Module . . . 36
4.5.1 Structure . . . 36
4.5.2 The Companion Object and Factory Method . . . 37
4.5.3 IO Bundle Design . . . 37
4.6 The Testbench . . . 38
4.6.1 Why a Separate Testbench? . . . 38
4.6.2 Structure . . . 38
4.7 Blackbox Verilog and Verilog Resources . . . 39
4.8 Coding Style . . . 39
4.8.1 What scalafmt and scalafix Handle . . . 40

4.8.2 What the ADR System Handles . . . 40
4.8.3 Scaladoc . . . 41
4.9 A Note on Internal Implementation . . . 42

5 Verifying Your Core **43**
5.1 The Verification Philosophy . . . 43
5.2 The Verification Architecture . . . 43
5.3 The ChiselTest Framework . . . 44
5.4 Directed Tests . . . 45
5.4.1 The Backend Annotations . . . 45
5.4.2 The Test Sequence . . . 45
5.4.3 Constrained-Random Techniques . . . 45
5.4.4 The Three Required Configurations . . . 46
5.4.5 Testing Require Constraints . . . 46
5.4.6 Test Documentation . . . 47
5.5 Code Coverage . . . 47
5.5.1 Interpreting Coverage Results . . . 48
5.5.2 Coverage Exclusions . . . 48
5.6 Formal Verification . . . 48
5.6.1 Why Formal Verification? . . . 48
5.6.2 The Formal Toolchain . . . 49
5.6.3 The Formal Testbench . . . 49
5.6.4 Temporal Primitives . . . 50
5.6.5 Property Types . . . 50
5.6.6 The Formal Test Runner . . . 51
5.6.7 What Good Formal Coverage Looks Like . . . 52
5.6.8 If You Are Not Implementing Formal Verification . . . 52
5.7 Running the Verification Suite . . . 52
5.7.1 Interpreting test.rpt . . . 53
5.7.2 Common Verification Pitfalls . . . 53

6 Documenting Your Core **54**
6.1 The Documentation Philosophy . . . 54
6.2 The User Guide . . . 54
6.2.1 Version History . . . 55
6.2.2 Errata and Known Issues . . . 55
6.2.3 Theory of Operations . . . 55
6.2.4 Port Descriptions . . . 56
6.2.5 Parameter Descriptions . . . 56
6.2.6 Simulation . . . 57
6.2.7 Synthesis . . . 57
6.3 The CHANGELOG . . . 58
6.4 The README . . . 59
6.5 Scaladoc . . . 59
6.6 License Documentation . . . 60
6.7 Authorship and Provenance . . . 60
6.7.1 The AUTHORSHIP.md File . . . 60
6.7.2 Recording Incorporated Code . . . 61

6.7.3 Upstream NOTICE Files . . . 62
6.7.4 Mixed-License Repositories . . . 62
6.7.5 When in Doubt, Write It Yourself . . . 63
6.8 Checking Documentation Compliance . . . 63

7 Running the Regression **64**
7.1 The make all Sequence . . . 64
7.2 Running Individual Targets . . . 64
7.3 Interpreting Results . . . 65
7.3.1 The Error Report . . . 65
7.3.2 Automatic Browser Launch . . . 66
7.3.3 The GitHub Actions Workflow . . . 66
7.4 Preparing for Certification Submission . . . 67

8 Submitting for Certification **68**
8.1 Before You Submit . . . 68
8.2 The Certification Workflow . . . 68
8.3 The ADR Compliance Report . . . 69
8.3.1 Reading the Report . . . 69
8.3.2 Verification Mechanism . . . 71
8.3.3 False Positives and Human Review . . . 71
8.4 Resolving ADR Findings . . . 71
8.4.1 Scala and Chisel Style Findings . . . 71
8.4.2 Documentation Consistency Findings . . . 72
8.4.3 Coverage Findings . . . 72
8.4.4 Synthesis Findings . . . 72
8.5 After Certification . . . 72
8.5.1 Citing chiselWare in Academic Work . . . 73

9 Maintaining Your Core **74**
9.1 Versioning Your Releases . . . 74
9.1.1 When to Increment Each Position . . . 74
9.1.2 Practical Examples . . . 74
9.1.3 Updating Version Records . . . 75
9.2 The Recertification Process . . . 75
9.3 Responding to User Issues . . . 76
9.4 Keeping Up with New CWS Versions . . . 76
9.4.1 Updating the SSE . . . 77
9.5 Deprecating and Retiring a Core . . . 77
9.6 Transferring Ownership . . . 77

Appendices **80**

A chiselWare Standard Reference **80**

B Standard Software Environment Deployment Guide **81**

C CI/CD Architecture **82**

D Scala and Chisel Resources **83**
D.1 Learning Scala . . . 83
D.1.1 Official Documentation . . . 83
D.1.2 Learning Materials . . . 83
D.1.3 Build Tools . . . 84
D.2 Learning Chisel . . . 84
D.2.1 Official Documentation . . . 84
D.2.2 Learning Materials . . . 84
D.2.3 Verification . . . 85
D.3 Open Source EDA Tools . . . 85
D.3.1 Simulation . . . 85
D.3.2 Synthesis and Timing . . . 85
D.3.3 Documentation . . . 85
D.4 chiselWare Community . . . 86

E Glossary **87**

Chapter 1

Introduction

Chip design is complex. Even the smallest designs involve thousands of decisions. In fact, one could argue that chip design is mostly about making decisions. Having a methodology and guidelines dramatically reduces the number of decisions a designer must make and establishes consistency and quality across the work of many teams.

In the mid-1990s, Verilog was a relatively new language and there were no guidelines as to how to design semiconductor IP. As a result, the nascent IP market struggled in terms of quality which held back the widespread adoption of IP reuse for semiconductor design. Semiconductor companies were eager to develop their own IP portfolios for internal reuse yet lacked a methodology to achieve this.

In 1998, Synopsys and Mentor Graphics published the Reuse Methodology Manual (RMM) which largely came from Synopsys' internal strict guidelines for creating their DesignWare IP portfolio. The publication of the RMM was a seminal event that eventually led to the multi-billion dollar market for the IP that powers today's semiconductor industry.

The RMM did not prescribe a Synopsys-way of doing things. Instead, it advocated for companies to establish their own methodology based on their needs and then enforce that discipline across their entire organization. It provided many examples of things that should be standardized and how to standardize them — naming conventions, for example. The naming convention itself was far less important than that there *was* a naming convention that everyone followed and that management would enforce.

chiselWare® is exactly this — a methodology and framework for developing professional-grade semiconductor IP using an open-source EDA toolchain and the Chisel hardware description language. This guide — the chiselWare Developers Guide (CWDG) — is your practical companion for building, verifying, documenting, and certifying a chiselWare Core.

1.1 Why Chisel?

The short answer is that Verilog is a terrible language for describing hardware.

Verilog was never designed to be a language of design. It was invented in the 1980s as a simulation modeling language and quickly caught on. Its primacy in synthesis arose in 1988 when Synopsys developed logic synthesis and needed a language to ingest for RTL-to-gate synthesis. Verilog was

pervasive enough that Synopsys engineers realized they could apply a synthesis semantic to it, allowing a single language to be used for both synthesis and simulation. If you were a digital designer in 1992, it was brilliant. If you are a designer in 2026, you wonder: "What were they thinking?"

VHDL and SystemVerilog are hardly better. VHDL was originally a documentation language adapted for synthesis following the Verilog precedent. SystemVerilog's primary impetus was to support advanced verification techniques such as the Universal Verification Methodology (UVM). Neither was designed from the ground up as a language for *describing* hardware.

The deficiencies of Verilog are evident from the existence of an entire EDA market segment dedicated to addressing its weaknesses — linting tools such as Synopsys' SpyGlass exist precisely because Verilog cannot catch its own errors. The commercial motivation to perpetuate Verilog means that if change is to come, it must come from outside the commercial arena.

In 2012, the University of California at Berkeley released Chisel, an expressive hardware description language that generates Verilog.[1] Chisel inherits many features from Scala, a powerful modern software language that provides procedural, object-oriented, and functional programming paradigms. These capabilities in the hands of a hardware engineer provide a modern language capable of efficiently describing complex hardware designs. The result is hardware description that looks and feels like software development — with all the tooling, testing, and automation benefits that entails.

1.2 Why chiselWare?

Chisel solves the language problem. chiselWare solves the methodology problem.

A language alone does not produce quality IP. The RMM taught us that methodology — consistent conventions, required deliverables, automated checking — is what transforms a capable language into a reliable IP ecosystem. chiselWare provides that methodology for the Chisel community, not for a single corporation but for a distributed community of developers worldwide.

Fortunately, Chisel itself eliminates many of the rules that the RMM needed to address. The semantic weaknesses of Verilog — non-blocking assignment hazards, unintended latches, combinational loops — simply do not exist in Chisel. So the chiselWare methodology can focus on what matters: consistency, quality, and trust.

IP cores developed in compliance with the chiselWare Standard (CWS) qualify for inclusion in the chiselWare Core Registry and entitle their authors to use the chiselWare logo. Core Users can rely on a certified chiselWare Core meeting a defined standard of quality, completeness, and consistency — regardless of who developed it.

1.3 The Three Tenets

There are three overarching principles behind chiselWare, in this order of priority:

[1] Chisel can generate either Verilog or SystemVerilog.

1.3.1 Be Kind to the Core User

Commercial IP users are not hobbyists. They have no interest in reading your source code. They are interested in *using* your core. As with other high-quality commercial IP, the user should never need to read the source code to understand what the core does or how to use it.

chiselWare achieves this through consistency. Every certified chiselWare Core delivers:

- A consistent coding style
- A consistent set of deliverables in consistent locations — a user of Core A will find Core B's deliverables in exactly the same places
- A consistent and verified minimum level of functional correctness
- Consistent, complete documentation sufficient to integrate and configure the core without reference to the source code

1.3.2 Be Kind to the Core Developer

Second only to being kind to Core Users is being kind to Core Developers.

If developing chiselWare cores is so onerous that developers are unwilling to meet the requirements, there will be no cores and no users. The chiselWare Administrator therefore abides by the following principles:

Minimize the rules. No rule should be arbitrary. Every requirement in the CWS exists for a clear reason related to producing quality cores that are easy to use. If a rule draws sustained criticism from the developer community as being onerous or unreasonable, it should be reconsidered. Equally, if the user community repeatedly encounters the same problem, the developer community has an obligation to address it.

Do not reinvent the wheel. Where excellent conventions already exist — the Scala Style Guide, the Chisel Developer's Style Guide, Semantic Versioning, Keep a Changelog — chiselWare adopts them rather than inventing alternatives.

1.3.3 Consistency through Automation

There is an old saying from the days following the RMM:

> A rule without a tool is not a rule.

chiselWare agrees completely. It is unreasonable in the era of CI/CD to expect developers to memorize rules and manually verify compliance. Every normative requirement in the CWS has a corresponding automated check. This is not just a convenience — it is a design principle. The CWS explicitly does not admit requirements that cannot be automatically verified.

Automated checking enforces consistency across all chiselWare Cores and gives developers fast feedback during development rather than surprises at certification time.

1.4 The chiselWare Ecosystem

chiselWare is more than a standard. It is an ecosystem of tools, infrastructure, and community that supports the full lifecycle of a chiselWare Core from initial development through certification and

distribution. Understanding the ecosystem before you start will save you considerable time.

1.4.1 The chiselWare Standard (CWS)

The CWS is the normative companion to this guide. It defines every **shall** requirement that a chiselWare Core must meet for certification. When this guide says "the CWS requires" or references a CWS section, it is pointing you to the authoritative source.

This guide tells you *how* to meet the requirements. The CWS tells you *what* the requirements are. In the event of any conflict between the two, the CWS takes precedence.

1.4.2 The chiselWare Core Registry

The chiselWare Core Registry is the authoritative database of all certified chiselWare Cores. It is maintained directly by the IP Factory® — Core Developers do not interact with the Registry directly. All Registry updates are driven automatically by actions taken through the IP Factory, ensuring that the Registry always reflects the authoritative state of the certification process.

When a core achieves certification, the IP Factory updates the Registry automatically with the following information:

- The core name and functional description
- The certification status and Certification Version
- The applicable license type — Apache 2.0, CW-GL, or CW-CL
- The associated Organization ID and Team ID
- The contact details for license inquiries where applicable

Cores that have been certified under a prior version of the CWS remain in the Registry and retain their chiselWare Core status. They are marked with their Certification Version so that Core Users can make informed decisions about which version of a core meets their needs.

The chiselWare Core Registry is accessible at `https://chiselware.org`.

Important: The IP Factory is under development and is not publically available at the time of this writing. However, the description here (and thoughout) accurately describes its role in the chiselWare ecosystem.

1.4.3 The Standard Software Environment (SSE)

The SSE is the version-pinned toolchain against which all chiselWare Cores are developed and certified. It includes Chisel, Scala, sbt, simulation tools, synthesis tools, documentation tools, and code quality tools — all at specific pinned versions chosen to work together without conflict.

The SSE is distributed as a Docker image and as a pre-configured Azure VM. Organizations with internal IT constraints can also build the SSE from the published toolchain specification in Annex D of the CWS. The SSE eliminates the version incompatibility problems that plague ad hoc Chisel development. See Appendix B for detailed deployment instructions.

1.4.4 The Automated Design Review (ADR)

The ADR is the automated conformance checking system that verifies submitted cores against every requirement in the CWS. It combines deterministic static analysis tools with large language model (LLM) based analysis to evaluate conformance across coding style, documentation quality, synthesis deliverables, and more.

ADR results are delivered as a structured HTML compliance report with pass/fail status for each rule. Developers can run a local pre-check before formal submission to identify and fix issues early. The ADR system may produce false positives — the CWS provides a human review and waiver process for disputed findings. See Chapter 8 for the full certification and ADR submission workflow.

1.4.5 IP Factory

The IP Factory is the operational hub of the chiselWare ecosystem. It is the primary interface through which Core Developers and Core Users interact with chiselWare as a platform. Everything that makes chiselWare a functioning commercial ecosystem — registration, repository management, access control, licensing, and commerce — runs through the IP Factory.

Authentication and Vetting

The IP Factory uses LinkedIn's authentication API for all user registration. This is a deliberate choice: LinkedIn identity provides a meaningful level of professional vetting that anonymous account creation cannot. Core Developers and Core Users of paid cores are real, identifiable professionals — not anonymous actors. This matters in a commercial IP ecosystem where trust, accountability, and license compliance are essential.

Core Developer Registration and Project Management

For Core Developers, the IP Factory is where the chiselWare journey begins. After authenticating via LinkedIn, a developer registers their Developer ID, executes the chiselWare Developer Agreement, and registers or joins an Organization and Team. When a developer is ready to start a new core project, they submit a project request through the IP Factory. The IP Factory then provisions a new private repository under the chiselWare GitHub organization, pre-populated with the `00-000-dff` template, and assigns the Organization and Team IDs that will form the core's package namespace.

The IP Factory manages the full lifecycle of the repository — from private development through the certification process to public release. During development and certification, the repository remains private and accessible only to the Core Developer and the chiselWare Administrator. Upon successful certification, the IP Factory controls the transition to public visibility for open source cores or manages access control for commercial and government cores.

The chiselWare Core Registry is maintained directly by the IP Factory. When a core achieves certification, the IP Factory updates the Registry automatically with the core's certification status, Certification Version, license type, and associated Organization and Team IDs. Core Developers do not interact with the Registry directly — all Registry updates are driven by actions taken through the IP Factory, ensuring that the Registry always reflects the authoritative state of the certification process.

Core User Access and License Management

For Core Users, the IP Factory is the storefront and delivery mechanism for chiselWare Cores. Open source cores are freely accessible to any registered user. Commercial cores licensed under CW-CL require the Core User to execute a license agreement through the IP Factory before access is granted. Government cores licensed under CW-GL are subject to additional vetting and access controls administered through the IP Factory in accordance with applicable regulations.

License agreements are executed and stored within the IP Factory, providing an auditable record of every license grant. The IP Factory enforces access control at the repository level — a Core User who has not executed the required license agreement cannot access the core's repository or deliverables.

Commerce

The IP Factory integrates with Stripe for all commercial transactions. License fees for CW-CL cores, platform fees, and any other commercial arrangements are handled through Stripe's payment infrastructure. Core Developers who commercialize their cores through the IP Factory receive payouts via Stripe. The chiselWare Administrator retains a platform fee as defined in the chiselWare Developer Agreement.

Core Marketplace

The IP Factory serves as the public marketplace for discovering and evaluating certified chiselWare Cores. For Core Users, the marketplace is the primary entry point into the chiselWare ecosystem — a curated, searchable catalog of certified cores backed by the quality assurance of the CWS.

Discovery

The IP Factory automatically extracts key information from each certified core's repository to populate its marketplace listing. This includes the core name, functional description, parameter summary, port count, certification version, license type, and Organization details. Core Developers do not need to maintain a separate marketing page — the repository itself, kept compliant with the CWS, is the source of truth for the marketplace listing.

Core Users can discover cores through:

- **Full-text search** — search across core names, descriptions, and documentation content
- **Filters** — narrow results by license type, interface type, parameter range, certification version, or Organization
- **Browsing** — explore the full catalog sorted by rating, recency, or popularity

Ratings and Reviews

Every certified chiselWare Core listing includes a user rating system that helps Core Users make informed decisions. Ratings are visible during browsing and search, giving well-regarded cores the visibility they deserve.

To maintain the integrity of the rating system, the IP Factory enforces one important rule: **a Core User may not rate a core unless they have downloaded it**. This prevents uninformed ratings and ensures that every rating reflects actual experience with the core.

The chiselWare Quality Signal

The marketplace listing makes the chiselWare certification status prominent and visible. A Core User browsing the marketplace can immediately see whether a core is certified, under which CWS version it was certified, and whether a newer CWS version has been released since certification. This transparency gives Core Users the information they need to assess the currency and quality of a core before committing to it.

For Core Developers, a strong marketplace presence — driven by good ratings, clear documentation, and current certification — is the primary mechanism for building reputation within the chiselWare community. The investment in meeting CWS requirements pays dividends in marketplace visibility.

The Factory Metaphor

The name IP Factory is intentional. Just as a manufacturing facility takes raw materials and produces finished goods to a defined quality standard, the IP Factory takes Chisel source code and produces certified, packaged, commercially-ready semiconductor IP. The regression harness, ADR system, and certification process are the production line. The chiselWare Core Registry is the finished goods inventory. The IP Factory is the factory floor that ties it all together.

Every certified chiselWare Core generates a standard set of IP Factory deliverables as part of its regression harness — the generated netlist, core metadata, User Guide PDF, constraints file, and Verilog file list — packaged in a format that IP Factory can ingest directly. This is why the `ipf` target exists in every chiselWare Makefile. When you run `make all`, you are not just running tests — you are producing a finished product ready for the factory floor.

1.4.6 The 00-000-dff Template Repository

The fastest way to start a new chiselWare Core is to clone the `00-000-dff` template repository. It contains a complete, fully certified chiselWare Core implementing a simple D flip-flop — the simplest possible hardware design — as a working example of every CWS requirement met in practice.

`https://github.com/chiselWare/00-000-dff`

Everything in the template is designed to be customized for your core. Chapter 3 walks you through exactly what to change and what to leave alone.

1.5 How to Use This Guide

This guide is organized as a tutorial that follows the natural workflow of chiselWare Core development:

- **Part I** (Chapters 1–2) — Understand chiselWare and set up your development environment
- **Part II** (Chapters 3–7) — Build, verify, document, and regression-test your core

- **Part III** (Chapters 8–9) — Submit for certification and maintain your certified core
- **Appendices** — Deep dives on the SSE, CI/CD architecture, and reference material

If you are starting a new core from scratch, read this guide from beginning to end. If you are returning to a specific topic, the appendices and CWS cross-references will help you find what you need quickly.

Throughout this guide, normative requirements are not restated — they belong in the CWS. Instead, each chapter explains the thinking behind the requirements and shows you how to meet them efficiently using the tools and templates provided.

Chapter 2

Before You Begin

This chapter covers everything you need to have in place before you start developing a chiselWare Core. There are three things to sort out: your background knowledge, your development environment, and your registration with the chiselWare ecosystem. None of these take long, but skipping any of them will cost you more time later.

2.1 What You Should Already Know

This guide assumes working familiarity with the following. If any of these are gaps, the resources in Appendix D will help you get up to speed quickly.

2.1.1 Scala

chiselWare Cores are written in Scala using the Chisel library. You do not need to be a Scala expert — most chiselWare development uses a relatively small subset of the language — but you should be comfortable with:

- Case classes and companion objects
- Basic functional programming idioms — `map`, `filter`, `fold`
- The sbt build tool — compiling, testing, and publishing
- Scaladoc comment syntax

2.1.2 Chisel

You should be able to read and write basic Chisel modules. Specifically:

- Defining `Module` classes with `IO` bundles
- Combinational and sequential logic using `Wire`, `Reg`, and `RegNext`
- Parameterizing modules using case classes
- Writing basic ChiselTest testbenches

If you are new to Chisel, work through the Chisel Bootcamp before starting your first chiselWare Core. See Appendix D for the link.

2.1.3 Git and GitHub

All chiselWare Core development takes place in GitHub repositories managed by the IP Factory. You should be comfortable with:

- Cloning, branching, committing, and pushing
- Opening and reviewing pull requests
- Git tags and GitHub releases
- The feature branch workflow — creating a branch, doing work, opening a pull request

2.1.4 LaTeX

The chiselWare User Guide is written in LaTeX. You do not need deep LaTeX expertise — the template handles most of the structure — but you should be able to:

- Edit `.tex` files and compile them to PDF
- Add content to existing sections and tables
- Include figures and code listings

2.2 Setting Up the Standard Software Environment

The Standard Software Environment (SSE) is the version-pinned toolchain against which all chiselWare Cores are developed and certified. Before you write a single line of code, you need to be running inside the SSE. This is non-negotiable — cores developed outside the SSE cannot be certified.

The SSE is defined normatively in Annex C of the CWS. That annex is the authoritative reference for which tools are included and at which versions. This section covers how to get it running.

The SSE is available in three forms. Choose the one that fits your situation.

2.2.1 Docker Image (Recommended)

The Docker image is the fastest way to get the SSE running and is the recommended approach for individual developers. It requires Docker Desktop or Docker Engine on your machine.

Everything you need is in the chiselWare DevKit repository:

https://github.com/chiselWare/devkit

Clone it once:

```
git clone https://github.com/chiselWare/devkit.git
```

Inside `docker/` you will find three launch scripts:

Script	Platform
`run-chiselware-linux.sh`	Linux
`run-chiselware-mac.sh`	macOS (Apple Silicon)
`run-chiselware-wsl.sh`	Windows (WSL2)

Use these rather than invoking `docker run` by hand — they handle SSH key mounting, X forwarding, and workspace binding for you.

Rather than copying a script into each project, add the DevKit `docker/` directory to your `PATH` so the scripts are available from any project directory. Add a line like this to your shell profile (`.bashrc`, `.zshrc`, or equivalent):

```
export PATH="$PATH:$HOME/devkit/docker"
```

Adjust the path to wherever you cloned the DevKit. With that in place, pull the image once and launch from any project directory:

```
docker pull ghcr.io/chiselware/dev-full:x.y.z
cd ~/my-chisel-project
run-chiselware-mac.sh -v <x.y.z>
```

The directory you launch from becomes `/workspace` inside the container, so always launch from your project root. Files written outside `/workspace` are lost when the container exits.

Using VS Code

If you use VS Code, you can develop directly inside the SSE container with SSH keys, Git configuration, and Scala language support set up for you. Every chiselWare Core repository is provisioned with a `.devcontainer/` configuration already in place, so once you clone your repository you can reopen it inside the container with a single command.

See `DOCKER.md` in the DevKit for details on using VS Code with the containers, including Dev Containers and browser-based `code` tunnels.

2.2.2 Azure Virtual Machine

The Azure VM is a pre-configured cloud environment suitable for teams who prefer a shared, managed development environment. It is available via the Azure Marketplace under the Rocksavage Technology, Inc. publisher account.

The VM provides the complete SSE toolchain pre-installed. No local Docker installation is required. See Appendix B for provisioning and connection instructions.

2.2.3 Self-Built Environment

Organizations with internal IT policies that prohibit external Docker images or cloud services — common in defense contractors and large semiconductor companies — can build the SSE from the published toolchain specification in Annex C of the CWS.

Your organization's CAD or IT team should use that specification as the authoritative reference for building and validating the environment. See Appendix B for guidance on self-built environment validation.

Unlike the Docker image and Azure VM — where the toolchain is pinned and guaranteed — a self-built environment carries a real risk of version mismatch. After building, confirm every tool is present and reports the expected version:

```
sbt --version
firtool --version
verilator --version
yosys -V
sta -version
z3 --version
sby --version
```

The versions reported shall match Annex C of the CWS. If any tool is missing or reports an unexpected version, resolve it before proceeding — a mismatched toolchain will produce regression failures that are difficult to diagnose later.

2.2.4 A Note on VS Code

If you use VS Code, there is a smoother path than launching the container from a terminal. Every chiselWare Core repository is provisioned with a `.devcontainer/` configuration already in place — when you clone your repository in Chapter 3 and open it in VS Code, you can reopen it directly inside the SSE container with SSH keys, Git configuration, and Scala language support configured automatically.

You do not need to set anything up for this now. Install Docker and the VS Code Dev Containers extension, and Chapter 3 will walk you through it once you have a repository to open.

2.3 Registering with chiselWare

Before you can start a chiselWare Core project you need to register with the chiselWare ecosystem through the IP Factory. Registration takes only a few minutes and needs to be done only once.

2.3.1 Step 1 — Authenticate with LinkedIn

Navigate to `https://chiselware.org` and select **Sign In**. The IP Factory uses LinkedIn authentication to verify your professional identity. You will be redirected to LinkedIn to authorize the chiselWare application. No LinkedIn data is stored beyond what is needed for identity verification.

2.3.2 Step 2 — Complete Your Developer Profile

After authenticating, complete your developer profile. This includes your name, contact email, and any relevant professional affiliations. Your profile information is associated with your Developer ID and will appear in the chiselWare Core Registry for cores you certify.

2.3.3 Step 3 — Execute the Developer Agreement

Before you develop a core you must execute the chiselWare Developer Agreement. The Agreement covers:

- Your rights and obligations as a Core Developer
- The terms under which cores may be submitted and distributed within the chiselWare ecosystem
- Your warranty that submitted cores do not infringe third-party intellectual property
- The platform fee structure for commercial cores

Read the Agreement carefully before executing it. If you are developing cores as part of your employment, check with your employer before executing — some employment agreements affect IP ownership.

2.3.4 Step 4 — Register or Join an Organization

Every chiselWare Core is associated with an Organization and a Team. You have three options:

Register a new Organization if you represent a company, university, or other institution that is not yet registered. The IP Factory will assign your Organization an Organization ID.

Join an existing Organization if your employer or institution is already registered. Contact your Organization's chiselWare administrator to be added to the appropriate Team.

Use the Independent Organization (`oIN`) if you are developing cores independently with no formal institutional affiliation, or if you are developing a core entirely on your own account that is separate from your employer's chiselWare presence. The Independent Organization is a reserved catchall provided by the chiselWare Administrator for exactly this purpose. Cores developed under `oIN` carry no implied affiliation with any other Organization — it is your sole responsibility to ensure that cores submitted under `oIN` do not contain intellectual property belonging to any other Organization.

Note that a developer may belong to more than one Organization simultaneously. For example, a university researcher may develop cores under their university's Organization as part of a sponsored research project while also developing independent cores under `oIN`.

2.3.5 Step 5 — Register or Join a Team

Within your Organization, cores are further organized by Team. A Team typically corresponds to a product group, research group, or development team within the Organization. The IP Factory assigns your Team a three-character Team ID.

Once you have a Developer ID, an Organization ID, and a Team ID, you are ready to request your first core project.

2.3.6 Step 6 — Request a Core Project

When you are ready to start a new core, submit a project request through the IP Factory. Provide:

- The proposed core name in lowercase using only `[a-z0-9-]` characters

- A brief functional description of the core
- The applicable license type — Apache 2.0, CW-GL, or CW-CL

The IP Factory will provision a new private repository under the chiselWare GitHub organization, pre-populated with the `00-000-dff` template, and notify you when it is ready. Your core's package namespace — `org.chiselware.cores.o<OrgID>.t<TeamID>.<corename>` — is assigned automatically based on your Organization ID, Team ID, and core name.

You are now ready to start developing. Chapter 3 walks you through your first steps in the repository.

Chapter 3

Getting Started with the Template

By the time you reach this chapter you have the SSE running and a provisioned repository waiting for you in the chiselWare GitHub organization. The IP Factory has pre-populated that repository with the `00-000-dff` template — a complete, fully certified chiselWare Core implementing a D flip-flop. Everything in the template is designed to be customized for your core. This chapter walks you through what the template contains, what you need to change, and how to verify that everything is working before you write a single line of your own RTL.

3.1 The 00-000-dff Template Core

The `00-000-dff` template is not just a skeleton — it is a fully working, fully certified chiselWare Core. Every CWS requirement is met. Every Makefile target works. The User Guide compiles to PDF. The synthesis runs and achieves timing closure.

This matters because it means your starting point is already correct. You are not building compliance from scratch — you are customizing a compliant core for your specific design. The template does the infrastructure work so you can focus on the hardware.

The D flip-flop was chosen deliberately as the simplest possible hardware design. It has one input, one output, one clock, one reset, and one parameter. There is nothing to distract you from understanding the infrastructure around it.

3.2 Cloning Your Repository

Your provisioned repository is available in the chiselWare GitHub organization. Clone it inside the SSE environment:

```
git clone git@github.com:chiselWare/<your-repo-name>.git
cd <your-repo-name>
```

The repository name follows the convention `<OrgID>-<TeamID>-<corename>`, for example `01-abc-mycore`. This name is assigned by the IP Factory at project registration and matches your core's package namespace.

Before making any changes, create a feature branch to work on:

```
git checkout -b feature/initial-setup
```

All development work in chiselWare happens on feature branches. You will never commit directly to `staging` or `main` — those branches are protected and managed by the CI/CD infrastructure. Getting into this habit from your very first commit will save you headaches later.

3.3 Tour of the Repository

Before making any changes, spend a few minutes understanding what you have. Run:

```
make list
```

This lists all available Makefile targets. Then take a look at the top-level structure:

```
.
+-- .github/
|   +-- workflows/
|       +-- regression.yml
+-- .scalafmt.conf
+-- .scalafix.conf
+-- .scalafix-test.conf
+-- .devcontainer
+-- .gitignore
+-- build.sbt
+-- AUTHORSHIP.md
+-- CHANGELOG.md
+-- LICENSE.md
+-- Makefile
+-- modules/
|   +-- dff/
|       +-- docs/
|       |   +-- user-guide/
|       |       +-- Dff.tex
|       |       +-- draw.io/
|       |       +-- errata.tex
|       |       +-- images/
|       |       +-- params.tex
|       |       +-- pdf/
|       |       +-- ports.tex
|       |       +-- sim.tex
|       |       +-- syn.tex
|       |       +-- tops.tex
|       |       +-- wavedrom/
|       +-- src/
|           +-- main/
|           |   +-- resources/
|           |   |   +-- stdcells.lib
|           |   |   +-- vsrc/
|           |   +-- scala/org/chiselware/cores/o00/t000/dff/
|           |       +-- Dff.scala
|           |       +-- DffParams.scala
|           +-- test/
|               +-- scala/org/chiselware/cores/o00/t000/dff/
|                   +-- DffFormalTb.scala  (optional)
|                   +-- DffFormalTest.scala (optional)
|                   +-- DffTb.scala
|                   +-- DffTest.scala
+-- project/
|   +-- build.properties
|   +-- metals.sbt
```

```
|   +-- plugins.sbt
+-- README.md
```

Here is what each piece does and whether you will need to touch it:

File or Directory	Touch?	Purpose
`Makefile`	Yes	Regression harness. Change items at the top of the file
`build.sbt`	Yes	sbt build definition. Update core name, version, and package
`modules/dff/`	Yes	Rename to your core name and replace contents
`docs/user-guide/`	Yes	Replace template content with your documentation
`AUTHORSHIP.md`	Yes	Update for your core
`CHANGELOG.md`	Yes	Update for your core
`README.md`	Yes	Update for your core
`LICENSE.md`	Yes	Update copyright holder and license type
`DffParams.scala`	Yes	Rename and replace with your parameter case class and companion object
`Dff.scala`	Yes	Rename and replace with your top-level module
`DffTb.scala`	Yes	Rename and replace with your simulation testbench
`DffTest.scala`	Yes	Rename and update with your test suite
`DffFormalTb.scala`	Yes	Rename and replace with your formal testbench, or delete if not implementing formal verification
`DffFormalTest.scala`	Yes	Rename and update with your formal test runner, or delete if not implementing formal verification
`.scalafmt.conf`	No	SSE-defined formatter configuration
`.scalafix.conf`	No	SSE-defined lint configuration
`.scalafix-test.conf`	No	SSE-defined test lint configuration
`.github/workflows/`	Minimal	GitHub Actions workflow. Change `module_name` only
`project/`	No	sbt project configuration. Leave as-is

The configuration files — `.scalafmt.conf`, `.scalafix.conf`, and `.scalafix-test.conf` — are distributed as part of the SSE and shall not be modified. They define the coding style rules that ADR will check your core against. Modifying them will not make your core compliant — it will make it non-compliant in ways that are harder to debug.

3.4 The Customizations

Getting from the Dff template to your own core requires the following customizations. Do these in order before writing any of your own RTL.

3.4.1 Customization 1 — Rename the Directories

Rename the `modules/dff` directory to your core name:

```
mv modules/dff modules/<corename> # Change module name
```

Where <corename> is the lowercase core name assigned by the IP Factory, using only characters in the set [a-z0-9-].

Also update the package path in the source directory structure to reflect your Organization ID and Team ID. The source files live at:

```
src/main/scala/org/chiselware/cores/o<OrgID>/t<TeamID>/<corename>/
src/test/scala/org/chiselware/cores/o<OrgID>/t<TeamID>/<corename>/
```

3.4.2 Customization 2 — Update the Makefile

The Makefile is the main regression harness that ties the core into the main chiselWare regression infrastructure. Only three customizations are required at the top of the Makefile. Note the CoreName is the UpperCamelCase version of the core name.

The last parameter is important for the regression harness to ensure that the .ipf/ directory is correctly generated for the IP Factory API. Enter the parameters associated with the Verilog code, such as bus widths, etc. Enter them as a single string with the key-value pairs separated by a comma as shown below.

```
#==== Core Configuration ============================================
ORG_ID := "<OrgID>"
TEAM_ID := "TeamID>"
CORE_NAME := "<CodeName>"
IPF_PARAMS := "param1=<value>, <param2=value>"
#==== End Core Configuration -- do not modify below this line =============
```

3.4.3 Customization 3 — Update build.sbt

Similar to the Makefile, the build.sbt file needs only a few customizations that are found at the top of the file. Note that here, the corename is fully lowercase.

```
//==== Core Configuration ============================================
  2 val coreNameLc = "<corename>"
  3 val coreVersion = "1.0.0"
  4 val orgId = "<OrgID"
  5 val teamId = "<TeamID>"
  6 val chiselWareVersion = "x.y.z" // chiselWare platform version used (from
     Maven)
  7 val spdxCopyrightText = Seq("<COPYRIGHT HOLDER>") // add as many as needed
  8 //==== End Core Configuration -- do not modify below this line =========
// -- End Core Configuration -- do not modify below this line ----------------
```

Set the coreVersion to 1.0.0 — this is the first release of your core and will be tagged as such after certification. You will have many pre-certification tags as you develop.

Set the chiselWareVersion to the release of the chiselWare binaries that will be used for your new core. New cores should use the latest version.

The supported versions can be found on Maven here:

https://central.sonatype.com/artifact/org.chiselware/chiselware-scalafix-rules_2.13/versions

Update the `spdxCopyrightText` with the copyright text that will be stamped on the generated Verilog files. For companies it would be something like this: `Acme, Inc.` or for individual developers something like `John Doe`. If you have multiple developers, you can can add then like this: `Seq("John Doe","Jane Doe")`.

Run `make conformance` to insure your customizations have not broken any of the template files.

3.4.4 Customization 4 — Update the GitHub Actions Workflow

Open `.github/workflows/regression.yml` and change the `module_name` input to your core name:

```
with:
  module_name: "<corename>"
```

Do not change anything else in the workflow file. The workflow is provided by the chiselWare shared CI framework and shall not be modified beyond this single field.

3.4.5 Customization 5 — Update README.md, LICENSE.md, CHANGELOG.md

In `README.md`, replace the Dff description with your core name and a brief functional description. Add your Organization ID and Team ID, and update the link to the User Guide PDF.

In `LICENSE.md`, add your copyright notice on the first line above the Apache 2.0 license text:

```
Copyright (c) <year> <Your Name or Organization>
```

The Apache 2.0 license text that follows shall remain unmodified. If your core uses CW-GL or CW-CL rather than Apache 2.0, replace the entire file content with the appropriate SPDX identifier and contact details as defined in CWS Section 12.5.2.

In `CHANGELOG.md`, replace the Dff entry with an initial entry for your core:

```
## [1.0.0] - YYYY-MM-DD
### Added
- Initial development version
```

3.4.6 Customization 6 — Rename and Update the Scala Files

There are four Scala files in the template that need to be renamed and updated for your core. These are the minimum files required for every chiselWare Core — your core will likely have additional files and subdirectories specific to your design.

Rename each file by replacing `Dff` with your `<CoreName>`:

- `DffParams.scala` → `<CoreName>Params.scala`
- `Dff.scala` → `<CoreName>.scala`
- `DffTb.scala` → `<CoreName>Tb.scala`
- `DffTest.scala` → `<CoreName>Test.scala`
- `DffFormalTb.scala` → `<CoreName>FormalTb.scala` (if using formal)

- `DffFormalTest.scala` → `<CoreName>FormalTest.scala` (if using formal)

The formal files are provided in the template as working examples. If you are not implementing formal verification for your core, delete both files before certification submission. Empty or placeholder formal files constitute a conformance failure — they must either contain real properties or be absent entirely.

Inside each file, replace all occurrences of `Dff` with your `<CoreName>` and update the package declaration to match your Organization ID and Team ID:

```
package org.chiselware.cores.o<OrgID>.t<TeamID>.<corename>
```

Also update the SPDX copyright header at the top of each file:

```
// SPDX-FileCopyrightText: <COPYRIGHT HOLDER>
// SPDX-License-Identifier: Apache-2.0
```

Run `make lint` after renaming to confirm there are no violations before proceeding.

3.4.7 Customization 7 — Configure the Parameter Companion Object

The `<CoreName>Params.scala` file contains not just the parameter case class but a companion object that drives the simulation and synthesis harness. This companion object requires careful attention — it is the control center for how your core gets tested and synthesized across multiple configurations.

Chapter 4 covers the full structure of `<CoreName>Params.scala` in detail. For now, the minimum changes needed to make the template work for your core are as follows.

Update `MainClassName`. This constant is used by the automation infrastructure to locate your top-level class. It must exactly match the UpperCamelCase name of your top-level module:

```
val MainClassName = "<CoreName>"
```

Update `simConfigMap`. This map defines the named configurations that the test harness will run automatically. At minimum it must contain three configurations covering your default, minimum, and maximum parameter settings:

```
val simConfigMap = LinkedHashMap[String, <CoreName>Params](
  "default" -> <CoreName>Params(),
  "minimum" -> <CoreName>Params(/* min values */),
  "maximum" -> <CoreName>Params(/* max values */)
)
```

The map keys become the configuration names reported in the test output and in the ADR compliance report. Additional configurations beyond the three required ones may be added at your discretion.

Update `synConfigMap`. This map defines the named configurations that Yosys will synthesize. At minimum it must also contain three configurations. The map keys use snake_case by convention since they appear in Verilog-facing synthesis scripts:

```
val synConfigMap = LinkedHashMap[String, <CoreName>Params](
  "small_default" -> <CoreName>Params(),
  "small_minimum" -> <CoreName>Params(/* min values */),
  "large_maximum" -> <CoreName>Params(/* max values */)
)
```

Update `SdcFile`. The `SdcFile` object generates the SDC timing constraints file for each synthesis configuration. Update the port names in the SDC data to match your core's IO bundle. The Dff template has these input and output delay constraints that you will need to adapt:

```
set_input_delay  -clock clock $inputDelay  {io_yourInputPort}
set_output_delay -clock clock $outputDelay {io_yourOutputPort}
```

Update `IpfParamsMap`. This map defines the parameter metadata that the IP Factory uses to display your core's configurable parameters in the marketplace. Every parameter in your case class must have a corresponding entry. Chapter 4 covers `IpfParamsMap` in detail.

3.4.8 Customization 8 — Update the Simulation Testbench

The `<CoreName>Tb.scala` simulation testbench resides in the `src/test/` directory. It instantiates your core and provides the simulation interface that `<CoreName>Test.scala` drives. Note that the testbench lives in `test/` rather than `main/` — it is a simulation artifact and is not synthesizable.

```
class <CoreName>Tb(p: <CoreName>Params) extends Module {
  val dut = <CoreName>(p)
  val io  = IO(new Bundle {
    // mirror the ports your tests need to access
  })
}
```

Note that inside `<CoreName>Tb`, which is itself a Chisel `Module`, you should use the factory method to instantiate your core:

```
val dut = <CoreName>(p)  // correct inside a Chisel Module
```

Do not use `Module(new <CoreName>(p))` at the call site as the factory method's `apply` method handles the `Module()` wrapper internally.

The only exception is `ChiselStage.emitSystemVerilog` in your `Main` object, as covered in Chapter 4.

3.4.9 Customization 9 — Update the Test Suite

The `<CoreName>Test.scala` file drives the testbench across all configurations defined in `simConfigMap`. The key pattern to preserve is the configuration loop at the top of the class:

```
for ((configName, config) <- <CoreName>Params.simConfigMap) {
  main(configName = configName, p = config)
}
```

This loop automatically runs your full test suite for every configuration in `simConfigMap`. When you add a new configuration to the map, it gets tested automatically without any changes to the test code itself. This is the mechanism that gives you composite code coverage across all configurations in a single regression run.

Replace the Dff-specific test logic in the `main` function with tests appropriate for your core. Also add an `assertThrows` test for each `require` constraint in your parameter case class:

```
it should "throw when <paramName> is out of range" in {
  assertThrows[IllegalArgumentException] {
    <CoreName>Params(<paramName> = <invalidValue>)
  }
}
```

Also update the `TargetDirAnnotation` path in the `backendAnnotations` sequence to match your core name:

```
TargetDirAnnotation("modules/<corename>/generated")
```

Chapter 5 covers the verification strategy and test writing in detail.

3.4.10 Customization 10 — Formal Verification (Optional)

The template includes two formal verification files: `DffFormalTb.scala` and `DffFormalTest.scala`. These are fully working examples of chiselWare formal verification and are provided as a reference implementation.

You have two choices:

If you are implementing formal verification — rename both files replacing `Dff` with your `<CoreName>`, update the package declaration, update the copyright header, and replace the Dff-specific properties in `<CoreName>FormalTb.scala` with properties appropriate for your core. The `FormalTest` tag and the `simConfigMap` iteration pattern in `<CoreName>FormalTest.scala` should be preserved unchanged. Chapter 5 covers formal verification in detail.

If you are not implementing formal verification — delete both files:

```
rm src/test/scala/.../DffFormalTb.scala
rm src/test/scala/.../DffFormalTest.scala
```

Do not leave the template formal files in place as placeholders. The ADR system checks for their presence and evaluates their content — placeholder files that contain only the Dff template properties constitute a conformance failure.

3.5 Running make all for the First Time

With all customizations complete, run the full regression (still with Dff code) to verify that your infrastructure is correctly set up before you start writing RTL:

```
make all
```

This will take several minutes on the first run as sbt downloads dependencies. On a successful run you will see each step of the regression sequence complete without errors and the final output will confirm:

On a successful run the final output will be:

ALL TESTS PASSED WITH NO ERRORS

If `make all` passes at this point, your infrastructure is correctly configured. You have a clean, compliant starting point. Everything from here is your core.

If `make all` fails, check `error.rpt` for details:

```
cat modules/<corename>/generated/error.rpt
```

3.6 The Branch Workflow

chiselWare repositories use a three-stage branch model that separates development, verification, and certification into distinct phases. Understanding this model before you start will save you from common workflow mistakes.

3.6.1 The Three Branches

feature/* branches are for development work. Create as many feature branches as you need and push commits freely. Feature branches are created from `main` and are never merged directly to `main`.

staging is the verification gate. A change is considered ready only after it has been merged into `staging` and the required CI checks pass. The authoritative regression — `make all` — runs automatically on every pull request targeting `staging`.

main is the certified branch. It reflects the last known-good, maintainer-approved state of the repository. It is updated only by promoting verified changes from `staging` via a pull request reviewed and approved by the chiselWare Administrator.

The key idea is that `staging` is where correctness is proven and `main` is where verified work is published.

3.6.2 The Development Workflow

Step 1 — Create a feature branch from main

Always start from the current certified baseline:

```
git checkout main
git pull
git checkout -b feature/my-feature
```

Make changes and commit normally. You can push your feature branch at any time — there is no CI trigger on feature branch pushes.

Step 2 — Run regressions locally while you develop

Run `make all` locally as you iterate. The CI system runs the same entrypoint, so if it passes locally it will behave consistently in `staging`:

```
make all
```

Use `make test` for faster feedback during active development and `make all` when you think you are ready to promote.

Step 3 — Open a pull request to staging

When your feature is ready, open a pull request from your feature branch to `staging`:

```
feature/my-feature -> staging
```

This triggers the authoritative CI regression. If checks fail, review the logs and artifacts — particularly `error.rpt` — to diagnose the issue. Fix it on your feature branch and push more commits. CI re-runs automatically on each new push.

Step 4 — Merge into staging

Once the pull request shows all required checks passing, merge it into `staging`. At this point the change is considered verified.

Step 5 — Promotion to main

The final step is a pull request from `staging` to `main`:

```
staging -> main
```

This is a pure promotion step — no additional regression runs here. The chiselWare Administrator reviews what is in `staging` and promotes it to `main` when appropriate. This keeps `main` clean, stable, and certified.

3.7 Committing Your Starting Point

Once `make all` passes, commit your customizations before writing any RTL:

```
git add .
git commit -m "Initial customization from 00-000-dff template"
git push origin feature/initial-setup
```

This gives you a clean starting point in your Git history that you can return to if needed. It also confirms that your feature branch is correctly connected to the remote repository and that your GitHub credentials are working inside the SSE environment.

You are now ready to start designing your core. Chapter 4 covers the parameter case class and its companion object in full detail, the top-level module, and everything you need to know about writing chiselWare-compliant Chisel.

Chapter 4

Designing Your Core

With a working infrastructure in place and a passing `make all`, you are ready to replace the Dff implementation with your own core. This chapter covers the Scala files that define your core's external interface and the design conventions that chiselWare requires you to follow. Internal implementation details are yours to decide. chiselWare only constrains the external interface, the coding style, and the automation hooks that connect your core to the regression harness and the IP Factory.

4.1 The Parameter Case Class

The parameter case class is the first thing you should write for any new chiselWare Core. It defines the complete set of parameters that a Core User can configure when instantiating your core, and it is the contract between you and your user about what is configurable and what is not.

4.1.1 Philosophy

Parameters are one of the most important aspects of a reusable IP core. A core with no parameters is a one-size-fits-all solution that rarely fits anyone perfectly. A core with too many parameters is a configuration nightmare. The goal is to expose the parameters that genuinely matter to Core Users while hiding implementation details that they should not need to know about.

Think about parameters from the Core User's perspective. They want to know: how wide is the data path? How deep is the FIFO? Does this core support high-speed mode? They do not want to know about internal pipeline stages, microarchitectural tradeoffs, or implementation choices.

4.1.2 Structure

Every chiselWare Core shall define exactly one top-level parameter case class: `<CoreName>Params`.

All parameters that influence Verilog generation or testbench behavior shall be members of this case class. No such parameters shall be defined outside it at the top level.

Here is the Dff parameter case class:

```
case class DffParams(
    width: Int = 1) {
  val widthMsg = "width must be greater than or equal 1"
  require(width >= 1, widthMsg)
}
```

Notice several things about this example:

- The class name is `DffParams` — the UpperCamelCase core name followed by `Params`
- Every parameter has a default value — Core Users can instantiate the core with no arguments and get a sensible default configuration
- Every bounded parameter has a `require` statement with an informative error message stored in a `val` so it can be referenced elsewhere in the companion object

4.1.3 Require Statements

Every parameter that has a defined minimum or maximum value shall have a corresponding `require` statement. This is one of the most important kindnesses you can show a Core User — a clear, informative error at elaboration time is infinitely better than a mysterious synthesis failure or a silently wrong circuit.

Store your error messages in named `val`s rather than inline string literals. This allows the same message to be referenced in the `IpfParamsMap` without duplication:

```
case class DffParams(width: Int = 1) {
  val widthMsg = "width must be greater than or equal 1"
  require(width >= 1, widthMsg)
}
```

The ADR system checks that every parameter documented with bounds in the User Guide has a corresponding `require` statement, and that the bounds match. Write your `require` statements and your User Guide parameter descriptions at the same time to keep them in sync.

4.1.4 Internal Parameterization

Below the top level you are free to use any Scala parameterization mechanism — implicit parameters, type parameters, abstract members, nested case classes, whatever serves your design best. chiselWare only constrains the top-level interface. Internal implementation is entirely your domain.

4.2 The Parameter Companion Object

The companion object of `<CoreName>Params` is one of the most important files in a chiselWare Core. It is the control center that connects your parameter definitions to the simulation harness, the synthesis harness, and the IP Factory. Understanding each component of the companion object is essential before you start development.

Here is the complete Dff companion object as a reference:

```
object DffParams {
  val MainClassName = "Dff"

  val simConfigMap = LinkedHashMap[String, DffParams](
    "config8"  -> DffParams(width = 8),
    "config32" -> DffParams(width = 32),
    "config64" -> DffParams(width = 64)
  )

  val synConfigMap = LinkedHashMap[String, DffParams](
    "small_1"   -> DffParams(width = 1),
    "medium_64" -> DffParams(width = 64),
    "large_128" -> DffParams(width = 128)
  )

  val synConfigs = DffParams.synConfigMap
    .map { case (configName, _) => configName }
    .mkString(" ")

  def fromMap(m: Map[String, String]): DffParams = {
    val width = m.get("width").map(_.toInt).getOrElse(1)
    DffParams(width = width)
  }
}
```

4.2.1 MainClassName

`MainClassName` is a string constant that identifies the top-level class name of your core. It is used the automation infrastructure to locate the correct class for Verilog generation and synthesis:

```
val MainClassName = "<CoreName>"
```

This must exactly match the UpperCamelCase name of your top-level Chisel module. A mismatch here will cause the Verilog generation step to fail with a confusing class-not-found error.

4.2.2 simConfigMap

`simConfigMap` is a `LinkedHashMap` that defines the named parameter configurations that the test harness will exercise. The test suite in `<CoreName>Test.scala` automatically iterates over every entry in this map and runs the full test suite for each configuration, producing composite code coverage across all configurations in a single regression run.

```
val simConfigMap = LinkedHashMap[String, <CoreName>Params](
  "default" -> <CoreName>Params(),
```

```
  "minimum" -> <CoreName>Params(/* min param values */),
  "maximum" -> <CoreName>Params(/* max param values */)
)
```

The map keys become the configuration names reported in the test output and in the ADR compliance report. Choose names that clearly communicate what each configuration represents.

At minimum, `simConfigMap` shall contain three configurations:

- **Default** — all parameters at their default values as defined in the case class
- **Minimum** — all parameters set to their minimum permitted values as defined by `require` constraints
- **Maximum** — all parameters set to their maximum permitted values as defined by `require` constraints

Additional configurations that exercise interesting or potentially problematic parameter combinations are encouraged and will improve your coverage numbers.

A `LinkedHashMap` is used rather than a plain `Map` to preserve insertion order — the configurations are tested in the order they appear in the map, which makes test output predictable and easier to read.

4.2.3 synConfigMap

`synConfigMap` defines the named parameter configurations that Yosys will synthesize. It follows the same structure as `simConfigMap` but uses snake_case keys by convention since these names appear in Verilog-facing synthesis scripts:

```
val synConfigMap = LinkedHashMap[String, <CoreName>Params](
  "small_default" -> <CoreName>Params(),
  "small_minimum" -> <CoreName>Params(/* min values */),
  "large_maximum" -> <CoreName>Params(/* max values */)
)
```

The synthesis configurations do not need to match the simulation configurations exactly, though they must also include at minimum a default, minimum, and maximum configuration. Synthesis configurations are typically named to reflect the expected silicon characteristics — `small_`, `medium_`, `large_` — rather than the abstract parameter values.

`synConfigs` is derived automatically from `synConfigMap` and provides the configuration names as a space-separated string for use by the Makefile:

```
val synConfigs = DffParams.synConfigMap
  .map { case (configName, _) => configName }
  .mkString(" ")
```

Do not modify `synConfigs` directly — it is computed from `synConfigMap` and will update automatically when you change the map.

4.2.4 fromMap

The `fromMap` method deserializes a `Map[String,String]` into a `<CoreName>Params` instance. It is called by the IP Factory when a Core User configures your core through the marketplace — the IP Factory passes the user's parameter selections as a string map and `fromMap` converts them to the correct Scala types:

```
def fromMap(m: Map[String, String]): <CoreName>Params = {
  val myParam = m.get("myParam").map(_.toInt).getOrElse(defaultValue)
  // add conversions for each parameter
  <CoreName>Params(myParam = myParam)
}
```

Add a conversion for every parameter in your case class. Use `getOrElse` with the parameter's default value so that parameters not specified by the Core User fall back to their defaults gracefully.

4.3 The SDC File Generator

The `SdcFile` object generates the Synopsys Design Constraints (SDC) file for each synthesis configuration. The SDC file specifies the timing constraints — clock frequency, input delays, and output delays — that Yosys uses during synthesis and that OpenSTA uses during static timing analysis.

Here is the Dff `SdcFile` object:

```
object SdcFile {
  def create(p: DffParams, sdcFilePath: String): Unit = {
    val period        = 5.000  // ns (200 MHz)
    val dutyCycle     = 0.50
    val inputDelayPct = 0.2
    val outputDelayPct = 0.2

    val inputDelay  = period * inputDelayPct
    val outputDelay = period * outputDelayPct
    val fallingEdge = period * dutyCycle

    val sdcFileData =
      s"""
      |create_clock -period $period \
      |  -waveform {0 $fallingEdge} clock
      |set_input_delay  -clock clock $inputDelay {reset}
      |set_input_delay  -clock clock $inputDelay {io_d}
      |set_input_delay  -clock clock $inputDelay {io_enable}
      |set_output_delay -clock clock $outputDelay {io_q}
    """.stripMargin.trim
    ...
  }
}
```

4.3.1 Setting the Target Frequency

The `period` variable defines the target clock frequency for synthesis and timing analysis. The default of 5.0ns corresponds to 200 MHz. Adjust this to reflect the target operating frequency documented in your User Guide Synthesis section — the CWS requires that timing closure is achieved at the frequency stated in the User Guide.

A tighter constraint will produce a more aggressively optimized netlist at the cost of longer synthesis runtime. A looser constraint will produce a faster synthesis run but may not reflect the core's actual performance potential.

4.3.2 Port Names in SDC

The port names in the SDC constraints must exactly match the port names as they appear in the generated Verilog. Chisel maps IO bundle port names to Verilog with the prefix `io_`. For example, a port named `dataIn` in your IO bundle becomes `io_dataIn` in the generated Verilog and in the SDC file.

Update the `set_input_delay` and `set_output_delay` lines to match your core's IO bundle:

```
set_input_delay  -clock clock $inputDelay {io_yourInputPort}
set_output_delay -clock clock $outputDelay {io_yourOutputPort}
```

A mismatch between SDC port names and Verilog port names will cause OpenSTA to report unconstrained ports, which will appear as errors in `timing_summary.rpt` and cause `make all` to fail.

4.4 The IP Factory Parameters Map

The `IpfParamsMap` object defines the parameter metadata that the IP Factory uses to present your core's configurable parameters in the marketplace. Every parameter in your case class must have a corresponding entry in `IpfParamsMap`.

Here is the Dff `IpfParamsMap`:

```
object IpfParamsMap {
  val params = Map(
    "width" ->
      IpfParams(
        fieldType = FieldType.Field,
        pickList  = List.empty,
        message   = Some(DffParams().widthMsg)
      )
  )
}
```

4.4.1 FieldType

The `FieldType` enumeration controls how the parameter is presented to the Core User in the IP Factory marketplace:

- `FieldType.Field` — a free-entry numeric or text field. Use for parameters that accept a continuous range of values such as data width or FIFO depth.
- `FieldType.Toggle` — a boolean on/off switch. Use for parameters of type `Boolean`.
- `FieldType.Pick` — a dropdown list of permitted values. Use for parameters that only accept specific discrete values.

4.4.2 pickList

The `pickList` field provides the list of permitted values for `FieldType.Pick` parameters. For `Field` and `Toggle` parameters, pass `List.empty`:

```
// Field -- free entry, no pick list
"dataWidth" -> IpfParams(FieldType.Field, List.empty,
  Some(MyParams().dataWidthMsg))

// Toggle -- boolean, no pick list
"useEcc" -> IpfParams(FieldType.Toggle, List.empty, None)

// Pick -- discrete values only
"cacheSize" -> IpfParams(FieldType.Pick,
  List("32", "64", "128"), None)
```

4.4.3 message

The `message` field provides the validation message shown to the Core User when they enter an invalid parameter value in the marketplace. Pass the same message string used in the `require` statement of your case class — this is why storing require messages in named `val`s is recommended:

```
message = Some(DffParams().widthMsg)
```

For parameters without validation messages (such as `Toggle` parameters), pass `None`.

4.5 The Top-Level Module

The top-level module is the hardware implementation of your core. It is the Chisel `Module` class that Core Users will instantiate in their designs. Like the parameter case class, it has a defined structure at the top level.

4.5.1 Structure

The top-level module shall be named `<CoreName>` and shall accept a single `<CoreName>Params` instance as its constructor argument. Here is the Dff top-level module:

```
class Dff(params: DffParams) extends Module {
  val io = IO(new Bundle {
    val d      = Input(UInt(params.width.W))
    val enable = Input(Bool())
    val q      = Output(UInt(params.width.W))
  })
```

```
  val q = RegNext(io.d, 0.U(params.width.W))
  io.q := q
}
```

4.5.2 The Companion Object and Factory Method

Every chiselWare Core shall provide a companion object for the top-level module. The companion object's `apply` method enables the factory method instantiation pattern that chiselWare requires. Importantly, the `Module()` wrapper belongs inside the `apply` method, not at the call site:

```
object Dff {
  def apply(params: DffParams): Dff = Module(new Dff(params))
}
```

With this companion object in place, Core Users and testbenches instantiate your core without the `new` keyword or the `Module()` wrapper:

```
// Inside a Chisel Module -- factory method handles Module() internally
val myDff = Dff(DffParams(width = 16))
```

However there is an important exception. `ChiselStage.emitSystemVerilog` controls its own elaboration context and must receive an unevaluated module using `new` directly — do not use the factory method here:

```
// Inside ChiselStage.emitSystemVerilog -- use new directly
ChiselStage.emitSystemVerilog(
  new Dff(configParams),
  firtoolOpts = Array(...)
)
```

The rule is straightforward:

- **Inside a Chisel `Module`** — use the factory method: `Dff(params)`. The `apply` method handles the `Module()` wrapper internally.
- **Inside `ChiselStage.emitSystemVerilog`** — use `new Dff(params)` directly.

  `ChiselStage` manages its own elaboration context and does not expect a pre-wrapped module.

This distinction is not obvious and has tripped up many Chisel developers. The companion object exists to make the common case — hardware instantiation inside a Module — clean and consistent. The `ChiselStage` exception is a consequence of how Chisel's elaboration infrastructure works internally.

4.5.3 IO Bundle Design

Your IO bundle defines the external interface of your core — the ports that Core Users connect to. A few guidelines:

Use descriptive port names. Port names appear in the generated Verilog, in the User Guide port table, and in the ADR compliance report. Names like `dataIn` and `dataOut` are better than `d` and `q` for anything other than the simplest cores.

Use UpperCamelCase for bundle names and lowerCamelCase for port names. This follows the Chisel style guide and is enforced by the ADR rule set.

Do not use snake_case. Snake_case port names violate CHISEL-011 and will fail ADR. The only exception is blackbox definitions where the port names must match the underlying Verilog.

Clock and reset are implicit. Do not add `clock` and `reset` to your IO bundle — Chisel provides these implicitly. They will appear in the generated Verilog automatically.

4.6 The Testbench

The `<CoreName>Tb.scala` testbench is a Chisel module that wraps your core for simulation. It is instantiated by `<CoreName>Test.scala` and provides the simulation interface that the test suite drives. The testbench is not synthesized — it exists only for simulation purposes.

4.6.1 Why a Separate Testbench?

Separating the testbench from the core serves several purposes. It keeps the synthesizable RTL clean — simulation models, memory models, and bus functional models that are needed for testing but not for synthesis live in the testbench rather than in the core. It also allows the testbench to instantiate multiple instances of the core, or to add monitoring logic, without affecting the core's generated Verilog.

4.6.2 Structure

At minimum the testbench instantiates your core and exposes its ports:

```
class <CoreName>Tb(p: <CoreName>Params) extends Module {
  val dut = <CoreName>(p)

  val io = IO(new Bundle {
    val in     = Input(UInt(p.width.W))
    val enable = Input(Bool())
    val out    = Output(UInt(p.width.W))
  })

  dut.io.d      := io.in
  dut.io.enable := io.enable
  io.out        := dut.io.q
}
```

For more complex cores the testbench may also instantiate:

- Memory models for cores with SRAM interfaces
- Bus functional models for cores with APB, AXI, or other bus interfaces
- Protocol checkers or monitors for interface verification

- Multiple instances of the core for interaction testing

These simulation-only components belong in the testbench, not in the core, to keep the synthesizable RTL uncontaminated.

4.7 Blackbox Verilog and Verilog Resources

Sometimes a small amount of hand-written Verilog is unavoidable — technology-specific latches, asynchronous logic primitives, vendor IP that has no Chisel equivalent, simulation models, or standard cell libraries required for synthesis. chiselWare accommodates this through the `src/main/resources/vsrc/` directory, which is the designated location for all Verilog resources associated with a core.

All Verilog and SystemVerilog files shall be confined to `vsrc/`. Verilog files found outside this directory constitute a conformance failure.

That said, all synthesizable RTL shall be generated from Chisel source. Do not attempt to wrap an existing Verilog design in a Chisel blackbox interface and present it as a chiselWare Core — this is explicitly contrary to the intent of the standard and will not be accepted for certification regardless of whether individual ADR rules pass.

If you need a Chisel blackbox, define it using `BlackBox` and place the corresponding Verilog implementation in `vsrc/`:

```
class MyLatch extends BlackBox {
  val io = IO(new Bundle {
    val d  = Input(Bool())
    val en = Input(Bool())
    val q  = Output(Bool())
  })
}
```

The Dff template includes a `stdcells.lib` file in `resources/` as an example of a synthesis library resource used by the Yosys synthesis flow. This file illustrates how non-blackbox Verilog resources are placed alongside the `vsrc/` directory.

4.8 Coding Style

chiselWare defines a single authoritative coding style that is built upon the Scala Style Guide and the Chisel Developer's Style.The chiselWare coding provides additional consistency and clarity by removing certain ambigities in the other coding guides.

The some examples of that clarifications are:

- Multi-line comments shall use Scaladoc notation
- Module instances shall use the factory method pattern
- Snake_case naming is not permitted in synthesizable code

The full normative coding style requirements are defined in CWS Section 10 and enforced by the ADR system.

4.8.1 What scalafmt and scalafix Handle

The majority of coding style requirements are enforced automatically before you ever reach the ADR system. `scalafmt` handles formatting — indentation, line length, spacing, and brace placement. `scalafix` handles lint rules — import ordering, naming conventions, and structural patterns.

Both tools run as part of `make lint` and produce a `lint.rpt` report. Any failures here will appear in `error.rpt` and must be resolved before `make all` can pass. Run `make lint` frequently during development rather than waiting until you think you are done. Fixing one lint warning at a time is much easier than fixing fifty at the end.

4.8.2 What the ADR System Handles

Beyond what `scalafmt` and `scalafix` can check, the ADR system evaluates conformance requirements that require deeper analysis — things like the consistency between your Scala source and your User Guide, the quality and completeness of your Scaladoc comments, the correctness of your synthesis deliverables, and coding patterns that static formatters cannot detect.

The ADR system produces a structured HTML compliance report for your core. The report is organized into six sections matching the six categories of ADR rules: Scala Coding Style, Chisel Coding Style, chiselWare Guidelines, Code Coverage, Synthesis Deliverables, and Documentation. Each rule shows a pass or fail status with an explanation of any violations found.

Here is an example of what the ADR compliance report looks like:



Figure 4.1: ADR compliance report summary showing pass/fail status for each of the six rule categories.

And here is an example of a detailed rule failure with the explanation the ADR system provides:

A complete description of every ADR rule, what it checks, and how to resolve failures is in Annex A of the CWS. Chapter 8 covers how to submit your core for ADR and how to interpret and respond to the results.

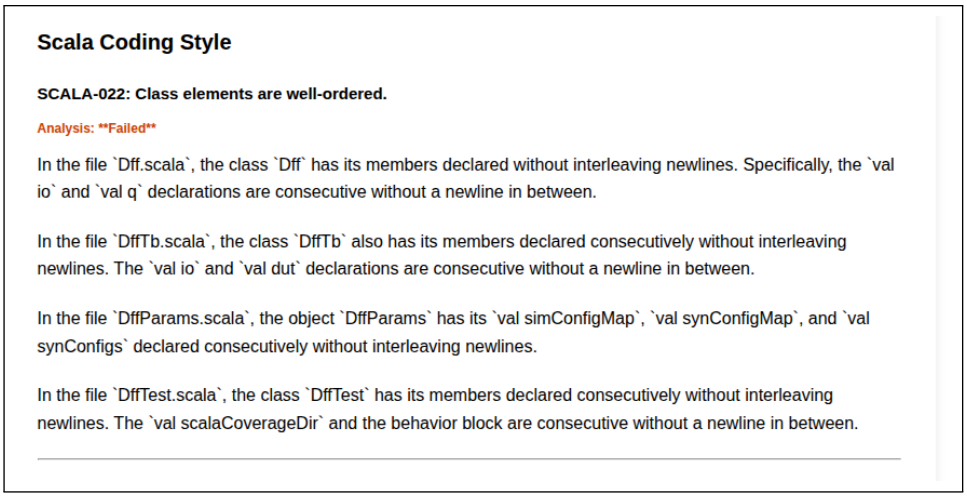


Figure 4.2: ADR compliance report detail showing a specific rule failure with explanation.

4.8.3 Scaladoc

Scaladoc comments deserve special attention because they serve two purposes in chiselWare: they document your code for future maintainers, and they are parsed by the ADR system to verify consistency with your User Guide.

The preferred Scaladoc style is a narrative preamble that explains *why* the code does what it does, not *what* it does line by line. The code itself shows what it does — the comment should provide context that the code cannot:

```
/** There is currently a limitation in Chisel with regard to
  * performing individual bit manipulation on a Vec of UInts.
  * This is due to the way Chisel generates hardware for Vecs,
  * which does not allow for individual bit manipulation on
  * the elements of the Vec.
  *
  * The workaround is to convert the Vec to a Vec of Bools,
  * perform the bit manipulation in Bool, and then convert
  * back to a Vec of UInts.
  */
val siBool: Vec[Bool] = Wire(Vec(numScanChains, Bool()))
```

Avoid inline comments that simply restate what the code does:

```
def flushBuffer(): Unit = {
  buffer.clear() // Clear the buffer
}
```

Use clear, descriptive variable names instead of comments where possible. A well-named variable rarely needs a comment:

```
// Poor -- requires a comment to understand
val w = params.d * params.n

// Better -- self-documenting
val totalBusWidth = params.dataWidth * params.numChannels
```

4.9 A Note on Internal Implementation

Everything in this chapter has focused on the external interface of your core — the parameter case class and its companion object, the top-level module, the testbench structure, and the coding style rules that apply to all Scala source files. These are the things chiselWare constrains.

Inside your core, below the top-level module, you are free to design however you see fit. Use whatever Chisel constructs, Scala patterns, or microarchitectural approaches best serve your design. chiselWare does not prescribe internal architecture, pipeline depth, state machine encoding, or any other implementation detail.

The one exception is coding style — the Scala and Chisel style rules apply to all source files, not just the top level. `scalafmt`, `scalafix`, and the ADR style checks see every file in your repository.

When you are happy with your design and it compiles cleanly, move on to Chapter 5 to write your test suite.

Chapter 5

Verifying Your Core

Verification is not an afterthought in chiselWare — it is a first-class deliverable treated with the same rigor as the RTL itself. A chiselWare Core without adequate verification is not a product; it is a prototype. This chapter covers what verification chiselWare requires, how to write effective tests using ChiselTest, and how to measure and interpret code coverage.

The good news is that Chisel makes verification significantly easier than Verilog. Your testbenches are written in Scala, which means you have the full power of a modern programming language — loops, functions, data structures, random number generation, and assertions — available in your testbench without the constraints of a hardware description language.

5.1 The Verification Philosophy

chiselWare takes a pragmatic approach to verification. The goal is not to prove mathematical correctness — that is the domain of formal verification, which chiselWare reserves for a future version of the standard when tooling matures. The goal is to give Core Users justified confidence that the core behaves correctly across its supported parameter space and input conditions.

Two principles guide the chiselWare verification approach:

Self-checking tests. Every test shall verify its own results. A test that drives inputs and observes outputs without asserting anything is not a test — it is a simulation. Use ChiselTest's `expect` method to assert expected output values explicitly. If an assertion fails, the test fails. If all assertions pass, you have evidence of correctness.

Coverage as a quality signal. Code coverage does not prove correctness, but low coverage is strong evidence of inadequate testing. chiselWare requires minimum coverage thresholds as a floor, not a ceiling. Achieving exactly 95% statement coverage and moving on is missing the point. Coverage is a tool for finding untested code, not a metric to optimize.

5.2 The Verification Architecture

chiselWare verification is built around three files that work together:

- `<CoreName>Params.scala` — defines the simulation configurations via `simConfigMap`

- `<CoreName>Tb.scala` — the testbench that instantiates the core and exposes its ports for testing
- `<CoreName>Test.scala` — the test suite that drives the testbench across all configurations automatically

The key insight is that the test suite does not need to know about individual configurations — it iterates over `simConfigMap` automatically. When you add a new configuration to the map, it gets tested without any changes to the test code. This produces composite code coverage across all configurations in a single regression run, which is both efficient and thorough.

5.3 The ChiselTest Framework

All chiselWare verification shall use the ChiselTest framework distributed as part of the SSE. ChiselTest provides a Scala-based testbench environment that integrates directly with Chisel's simulation infrastructure.

The Dff test suite illustrates the core ChiselTest patterns. Here is the structure of `DffTest.scala`:

```
class DffTest extends AnyFlatSpec
    with Matchers
    with ChiselScalatestTester {

  // Run the main test for each simulation configuration
  for ((configName, config) <- DffParams.simConfigMap) {
    main(configName = configName, p = config)
  }

  // Test that illegal configurations are caught
  behavior of "Configuration assertions"
  it should "throw when width is less than 1" in {
    assertThrows[IllegalArgumentException] {
      DffParams(width = 0)
    }
  }

  def main(configName: String, p: DffParams): Unit = {
    behavior of s"Dff directed tests (config: $configName)"
    it should "perform these tests successfully" in {
      test(new DffTb(p))
        .withAnnotations(backendAnnotations) { dut =>
          // test logic here
        }
    }
  }
}
```

Notice the structure: the configuration loop at the top calls `main` for each entry in `simConfigMap`, and `main` contains the actual test logic. This clean separation means adding a new configuration requires only a new entry in `simConfigMap` — the test code does not change.

5.4 Directed Tests

Every chiselWare Core shall include a suite of self-checking directed tests that verify the functional correctness of the core. Directed tests are deterministic — they drive specific inputs and assert specific outputs. They are the foundation of your verification suite.

5.4.1 The Backend Annotations

ChiselTest runs your tests through a simulation backend. The Dff template uses Verilator, which is the recommended backend for chiselWare:

```
val backendAnnotations = Seq(
VerilatorBackendAnnotation,
TargetDirAnnotation("modules/<corename>/generated")
)
```

The `TargetDirAnnotation` path was set during the initial template customization in Chapter 3. Verilator is the standard chiselWare simulation backend — the other backends shown as comments in the template are available but not required.

5.4.2 The Test Sequence

A well-structured directed test follows this sequence:

```
test(new DffTb(p))
  .withAnnotations(backendAnnotations) { dut =>

  // 1. Initialize all inputs to known values
  dut.io.enable.poke(false.B)
  dut.io.in.poke(0.U)

  // 2. Apply and release reset
  dut.reset.poke(true.B)
  dut.clock.step()
  dut.reset.poke(false.B)
  dut.io.out.expect(0.U)

  // 3. Exercise the core and assert outputs
  dut.io.enable.poke(1.U)
  dut.io.in.poke(42.U)
  dut.clock.step()
  dut.io.out.expect(42.U)
}
```

Always initialize inputs before applying reset, and always verify reset behavior before testing functional operation. A core that does not reset correctly is broken regardless of how well it behaves in normal operation.

5.4.3 Constrained-Random Techniques

The Dff test incorporates constrained-random verification directly into the directed test using Scala's random number generation:

```
for (i <- 1 to 10) {
  val myData = BigInt(p.width, scala.util.Random)
  dut.io.enable.poke(1.U)
  dut.io.in.poke(myData)
  dut.clock.step()
  dut.io.out.expect(myData)
  dut.io.enable.poke(0.U)
  dut.clock.step()
  dut.io.out.expect(myData)
}
```

`BigInt(p.width, scala.util.Random)` generates a random non-negative integer of up to `p.width` bits. This is particularly useful because it automatically scales with the parameter — a width-8 test generates 8-bit random values, a width-64 test generates 64-bit random values, without any changes to the test code.

Note that `scala.util.Random` without a fixed seed produces different values on every run. This is intentional for constrained random testing — you want to explore different areas of the input space on each regression run. If a test fails with random data, sbt will print the failing value in the output so you can reproduce it.

5.4.4 The Three Required Configurations

Because the test suite iterates over `simConfigMap` automatically, ensuring the three required configurations are covered is simply a matter of ensuring they are in the map:

```
val simConfigMap = LinkedHashMap[String, DffParams](
  "config8"  -> DffParams(width = 8),   // representative default
  "config32" -> DffParams(width = 32),  // mid-range
  "config64" -> DffParams(width = 64)   // maximum
)
```

The test suite runs the complete test logic for each of these configurations automatically, giving you coverage across the full parameter space in a single `make test` invocation.

5.4.5 Testing Require Constraints

Every `require` statement in your parameter case class shall have a corresponding `assertThrows` test that verifies the constraint is enforced. The Dff template shows the pattern:

```
behavior of "Configuration assertions"
it should "throw when width is less than 1" in {
  assertThrows[IllegalArgumentException] {
    DffParams(width = 0)
  }
}
```

`assertThrows` verifies that the expression inside it throws the specified exception type. If it does not throw, the test fails. This ensures that Core Users who misconfigure your core get a clear error message rather than a silent failure.

Add one `assertThrows` test for each `require` constraint in your case class. Test the boundary condition just outside the valid range — for a constraint of `width >= 1`, test with `width = 0`.

5.4.6 Test Documentation

Each directed test shall include a Scaladoc comment that specifies the purpose of the test, the configuration under which it runs, and the expected behavior:

```
/** Verifies that the Dff correctly registers its input on the
  * rising clock edge and holds the value when enable is low.
  *
  * Tests random data values across 10 iterations to exercise
  * the full width of the data path for the given configuration.
  *
  * Configuration: determined by simConfigMap entry
  * Expected: output matches input after one clock cycle when
  *           enable is high; output holds when enable is low
  */
def main(configName: String, p: DffParams): Unit = {
  ...
}
```

This documentation is parsed by the ADR system as part of the documentation conformance check. Keep it accurate and informative.

5.5 Code Coverage

Code coverage is measured by Scoverage and reported as part of every regression run. Two metrics are tracked:

- **Statement coverage** — the percentage of Scala statements executed during testing. chiselWare requires a minimum of 95%.
- **Branch coverage** — the percentage of conditional branches taken during testing. chiselWare requires a minimum of 90%.

Run coverage measurement with:

```
make cov
```

The coverage run executes the full test suite across all `simConfigMap` configurations, producing composite coverage numbers that reflect the combined effect of all configurations. When complete, `make cov` automatically opens the Scoverage HTML report in your browser:

```
modules/<corename>/generated/scalaCoverage/scoverage-report/index.html
```

This automatic browser launch is a deliberate reminder that the coverage report — like the User Guide PDF and the ADR compliance report — is a required deliverable that is regenerated on every `make all` run. Getting into the habit of reviewing it after every regression will help you catch coverage gaps early rather than discovering them at certification time.

5.5.1 Interpreting Coverage Results

Low statement coverage almost always means you are missing tests for specific code paths. Look at the red-highlighted lines in the Scoverage report and ask: under what input conditions would this line execute? Then write a directed test for that condition.

Low branch coverage often means you are not testing all combinations of conditional logic. A common cause is parameter-dependent `if` expressions that are only exercised in one configuration. Make sure your minimum and maximum configuration tests exercise both sides of every parameter-dependent branch.

The composite coverage across all configurations is what counts — a branch that is only reachable with maximum parameter values will show as uncovered if you only test the default configuration.

5.5.2 Coverage Exclusions

Occasionally a code region is genuinely untestable — dead code that exists for defensive purposes, or code that requires hardware fault injection to exercise. In these cases you may exclude the region from coverage measurement, but each exclusion shall be documented with a Scaladoc comment explaining the justification:

```
// $COVERAGE-OFF$ Defensive default case, unreachable in
// normal operation. All valid states are handled above.
case _ => io.out := 0.U
// $COVERAGE-ON$
```

The chiselWare Administrator reserves the right to reject coverage exclusion justifications that are not adequately reasoned. An exclusion that exists to inflate coverage numbers rather than to document a genuine untestable condition will not be accepted.

When your tests pass and your coverage meets the minimum thresholds, consider whether formal verification is appropriate for your core — see Section 5.6. When you are ready to move on, Chapter 6 covers writing your User Guide.

5.6 Formal Verification

5.6.1 Why Formal Verification?

Simulation tests the behavior of your core for specific input sequences that you thought to write tests for. Formal verification proves that specified properties hold for *all possible* input sequences up to a bounded depth. These are complementary approaches — simulation finds bugs you anticipated, formal finds bugs you did not.

For a simple core like a D flip-flop, formal verification may seem unnecessary. For cores with complex state machines, protocol interfaces, or subtle corner cases, formal verification provides a level of assurance that simulation alone cannot match. chiselWare makes formal verification accessible by integrating it directly into the familiar ChiselTest infrastructure.

Formal verification is optional in chiselWare. If you choose not to implement it, delete both formal files before certification submission. If you do implement it, the ADR system will evaluate your properties for completeness and correctness.

5.6.2 The Formal Toolchain

chiselWare formal verification uses three tools from the SSE:

- **`chiseltest.formal`** — the Chisel formal verification library that provides the `verify()` API and temporal primitives. Integrated directly with ChiselTest so formal and simulation tests coexist in the same project.
- **SymbiYosys (sby)** — the formal verification front-end that drives Bounded Model Checking (BMC) and induction proofs against the Verilog generated by Chisel.
- **Z3** — the SMT solver used by `chiseltest.formal` automatically. No annotation is needed in your test code — Z3 is selected by default.

See Appendix D for formal verification learning resources, including the ZipCPU blog which is the most practical reference available for hardware formal verification.

5.6.3 The Formal Testbench

Formal properties live in `<CoreName>FormalTb.scala` in the `src/test/` directory. The formal testbench follows the same structural pattern as the simulation testbench — it instantiates the core using `Module(new <CoreName>(p))` and adds properties — but uses `chiseltest.formal` primitives rather than ChiselTest simulation primitives.

Here is the structure of the Dff formal testbench as a reference:

```
class DffFormalTb(p: DffParams) extends Module {

  val io = IO(new Bundle {
    val d      = Input(UInt(p.width.W))
    val enable = Input(Bool())
  })

  val dut = Module(new Dff(p))
  dut.io.d      := io.d
  dut.io.enable := io.enable

  // assumptions
  // assume() -- constrain inputs to legal combinations

  // assertions
  when(past(reset.asBool)) {
    assert(dut.io.q === 0.U, "output must be zero after reset")
  }

  // cover points
  cover(io.enable, "enable high is reachable")
}
```

Note that the formal testbench IO bundle drives the DUT inputs as free variables — the formal tool explores all possible input combinations automatically. You do not need to write stimulus as you do in simulation.

5.6.4 Temporal Primitives

The following temporal primitives are available via `import chiseltest.formal._`:

Primitive	Description
`past(sig)`	Value of `sig` one cycle ago
`rose(sig)`	True when `sig` transitions 0→1
`fell(sig)`	True when `sig` transitions 1→0
`stable(sig)`	True when `sig === past(sig)`. Most useful for multi-field Bundle outputs where it replaces a field-by-field `past()` comparison

Two important rules for temporal primitives:

Reset casting. `reset` is of type `chisel3.Reset`, not `Bool` — always cast with `reset.asBool` before use in properties:

```
// Correct
when(past(reset.asBool)) { assert(...) }

// Wrong -- type mismatch
when(past(reset)) { assert(...) }
```

Registered outputs. For registered outputs use `past(rose(sig))` and `past(fell(sig))` rather than `rose(sig)` and `fell(sig)` directly. The effect of a clock edge on a registered output is visible one cycle after the edge:

```
// Correct -- effect visible one cycle after enable rises
when(past(rose(io.enable)) && !past(reset.asBool)) {
  assert(dut.io.q === past(io.d), "...")
}

// Wrong -- checks same cycle as edge, before register updates
when(rose(io.enable)) {
  assert(dut.io.q === io.d, "...")
}
```

5.6.5 Property Types

Three property types are available:

`assert()` — a property that must hold for all reachable states. A failing assert indicates a genuine bug in your core. Every formal testbench shall contain at least one assert.

`assume()` — a constraint on the environment. Use to restrict inputs to legal combinations when not all input combinations are valid for your core. Be conservative with assumptions — over-constraining the environment can hide real bugs.

`cover()` — a reachability check. If a cover point fails, the specified state is unreachable. Failing cover points often indicate over-constraining assumptions. Every formal testbench shall contain at least one cover point.

Note that `assert`, `assume`, and `cover` are Scala macros and do not support named parameters — the chiselWare named parameter convention does not apply to these three constructs.

5.6.6 The Formal Test Runner

Bounded Model Checking is run for `k` cycles after reset using `verify()` with `BoundedCheck(k)`:

```
class DffFormalTest extends AnyFlatSpec
    with ChiselScalatestTester
    with Formal {

  val bound = BoundedCheck(10)

  behavior of "Dff formal"

  for ((configName, config) <- DffParams.simConfigMap) {
    it should s"pass all formal properties ($configName)"
        taggedAs FormalTest in {
      verify(
        new DffFormalTb(config),
        Seq(
          bound,
          TargetDirAnnotation(
            s"modules/dff/generated/formal/$configName")
        )
      )
    }
  }
}
```

Choosing the Bound

`BoundedCheck(k)` proves properties for all input sequences of length up to `k` cycles after reset. Choosing an appropriate bound requires understanding your core's behavior:

- For simple combinational or single-cycle cores, `k=5` to `k=10` is typically sufficient
- For cores with FIFOs, pipelines, or multi-cycle protocols, `k` should be large enough to fill and drain the deepest structure at least once
- When in doubt, increase `k` until the proof runtime becomes impractical, then back off slightly

A bound that is too small may pass while missing real bugs. A bound that is too large increases proof runtime significantly. The Dff template uses `BoundedCheck(10)` as a reasonable starting point for a single-cycle core.

Running Formal Tests

Formal tests run as part of `make test/cov` alongside simulation tests. The `FormalTest` tag allows them to be run independently at the sbt level:

```
sbt "testOnly * -- -n FormalTest"  # formal only
sbt "testOnly * -- -l FormalTest"  # exclude formal
```

Formal test results appear in `generated/test.rpt` alongside simulation results. Formal failures are reported the same way as simulation failures — they appear in `error.rpt` and constitute a regression failure.

5.6.7 What Good Formal Coverage Looks Like

The Dff formal testbench in the template is a reference implementation that demonstrates the key patterns. A well-written formal testbench for any core should include:

- **Reset behavior** — Uses `past(reset.asBool)` to verify the core initializes to a known state after reset
- **Functional correctness** — verify the core's primary function using `past()` to relate outputs to prior inputs
- **Hold behavior** — verify outputs hold their values when the core is idle using `stable()` or `past()`
- **Edge-triggered behavior** — uses `past(rose())` and `past(fell())` to verify behavior at control signal transitions
- **Invariants** — verify properties that must hold every cycle without temporal operators, such as output range constraints
- **Cover points** — verify that key states are reachable, confirming your assumptions are not too restrictive

5.6.8 If You Are Not Implementing Formal Verification

Delete both formal files before certification submission:

```
rm src/test/scala/.../CoreNameFormalTb.scala
rm src/test/scala/.../CoreNameFormalTest.scala
```

Do not leave the template formal files in place as placeholders. The ADR system checks for their presence and evaluates their content — template files containing only the Dff properties constitute a conformance failure. The files must either contain real properties for your core or be absent entirely.

5.7 Running the Verification Suite

The complete verification suite runs as part of `make all` via the `cov` target. To run verification in isolation during development:

```
make test    # Run tests without coverage (faster)
make cov     # Run tests with coverage measurement (slower)
```

Use `make test` during active development for fast feedback. Switch to `make cov` when you want to check your coverage numbers or when preparing for certification submission.

5.7.1 Interpreting test.rpt

The `check` target scans `generated/test.rpt` for failures. A passing test run produces a line containing `failed 0` in the report. Any other failure count constitutes a regression failure and will appear in `error.rpt`.

If tests are failing, run `make test` directly and examine the sbt output for the specific assertion that failed. ChiselTest's failure output identifies the file, line number, and port name where the `expect` assertion failed, along with the actual and expected values:

```
[error] Assertion failed: io_out
[error]   Expected: 42
[error]   Actual:   0
[error]   at DffTest.scala:47
```

This gives you a precise starting point for debugging.

5.7.2 Common Verification Pitfalls

Off-by-one clock errors. The most common ChiselTest mistake is forgetting to step the clock before checking output. Sequential logic requires at least one clock edge to produce output. Always call `clock.step()` before `expect` on registered outputs.

Reset state assumptions. Do not assume your core starts in a known state without asserting reset explicitly. Always apply and release reset at the beginning of your test sequence before testing functional behavior.

Missing configuration coverage. If your `simConfigMap` only contains the default configuration, your coverage numbers will not reflect the full parameter space. Make sure your minimum and maximum configurations are in the map and that the tests exercise parameter-dependent code paths.

Ignoring coverage gaps. It is tempting to submit once you hit the coverage thresholds and move on. Resist this. Look at what is not covered and ask whether it matters. A gap in coverage for error handling code is more concerning than a gap in a dead default case.

Non-reproducible failures. If a constrained-random test fails, sbt prints the failing random value. Capture it and add a specific directed test for that value before moving on — it represents a real bug that could recur.

When your tests pass and your coverage meets the minimum thresholds, move on to Chapter 6 to write your User Guide.

Chapter 6

Documenting Your Core

Documentation is not a box to check at the end of development — it is a first-class deliverable that chiselWare treats with the same rigor as RTL and verification. A certified chiselWare Core shall be fully usable by a Core User without any reference to the source code. That is a high bar, and meeting it requires deliberate effort.

The good news is that the template provides all the structure you need. Your job is to fill it with accurate, clear content. This chapter walks you through each documentation deliverable, what it requires, and how to write it well.

6.1 The Documentation Philosophy

Good documentation is evidence that you understand your own core. A developer who cannot clearly explain what a port does or what a parameter controls probably does not fully understand the design themselves. Writing documentation forces that clarity.

Think about documentation from the Core User's perspective. They are a professional engineer with limited time. They want to know: what does this core do, how do I connect it, how do I configure it, and how do I verify that it works in my system. They do not want to read your source code to find out.

The ADR system enforces minimum documentation standards automatically — but meeting the minimum is not the goal. The goal is documentation that makes Core Users confident and productive.

6.2 The User Guide

The User Guide is the primary documentation deliverable for a chiselWare Core. It is written in LaTeX using the template provided in the `00-000-dff` repository. The template provides all required sections pre-structured — your job is to populate them with accurate content for your core.

Run `make docs` to generate the User Guide PDF and open it automatically in your browser:

```
make docs
```

Like the coverage report, this automatic browser launch is a reminder that the User Guide is a living deliverable regenerated on every `make all` run. Review it regularly during development — do not wait until you think you are done to look at the generated PDF.

The User Guide LaTeX source is organized into separate `.tex` files for each section, all included from the top-level `<CoreName>.tex` file:

```
modules/<corename>/docs/user-guide/
  <CoreName>.tex      Top-level file
  tops.tex            Theory of Operations
  ports.tex           Port Descriptions
  params.tex          Parameter Descriptions
  sim.tex             Simulation
  syn.tex             Synthesis
  errata.tex          Errata and Known Issues
```

6.2.1 Version History

The version history table records all released versions of your core, their release dates, and a description of changes. It lives in the top-level `<CoreName>.tex` file.

Update the version history table with each release. The version recorded here shall exactly match the version in `build.sbt` and the Git release tag — the ADR system checks this consistency automatically.

The date in the version history reflects when the documentation was completed, not when the Git tag was applied. These will typically differ since the certification process takes time after the documentation is written.

```
\begin{tabular}{lll}
  \hline
  \textbf{Version} & \textbf{Date} & \textbf{Description} \\
  \hline
  1.0.0 & 2026-01-15 & Initial certified release \\
  \hline
\end{tabular}
```

6.2.2 Errata and Known Issues

The errata section records known defects, limitations, and workarounds. It lives in `errata.tex` and shall be present even if empty. An empty errata section is not an embarrassment — it is a statement that no known issues exist at the time of release.

If issues are discovered after certification, update the errata section and release a Patch version. Do not leave known issues undocumented.

6.2.3 Theory of Operations

The Theory of Operations section lives in `tops.tex` and provides a functional description of your core. This is where you explain what the core does, how it works at a high level, and any important behavioral characteristics that a Core User needs to understand.

Write this section for an engineer who has never seen your core before. Include:

- A brief functional summary — one or two sentences that describe what the core does
- A block diagram if the core has meaningful internal structure
- Key behavioral characteristics — latency, throughput, reset behavior, clock domain requirements
- Any important limitations or design decisions that affect how the core is used
- Timing diagrams for interfaces that have non-obvious timing requirements

Block diagrams go in the `draw.io/` directory as `.drawio` files and are exported to PNG for inclusion in the document. Timing diagrams go in the `wavedrom/` directory as JSON files and are rendered to PNG. The template includes examples of both.

6.2.4 Port Descriptions

The port descriptions live in `ports.tex` and are one of the most important sections of the User Guide. Every port in your top-level IO bundle shall be documented here. The ADR system cross-references the port table against your Chisel source and flags any discrepancies — missing ports, extra ports, mismatched directions, or inconsistent widths all constitute conformance failures.

The template provides a port table structure. Fill in each row:

```
\begin{tabular}{llll}
  \hline
  \textbf{Port} & \textbf{Dir} & \textbf{Width} &
  \textbf{Description} \\
  \hline
  clock  & In  & 1           & System clock \\
  reset  & In  & 1           & Synchronous reset, active high \\
  io\_d  & In  & dataWidth   & Data input \\
  io\_q  & Out & dataWidth   & Registered data output \\
  \hline
\end{tabular}
```

A few guidelines for writing good port descriptions:

Be specific about timing. "Data input" is less useful than "Data input, sampled on the rising clock edge when enable is high" A Core User connecting your core needs to know when signals are sampled and when outputs are valid.

Document active levels. For control signals, always state whether they are active high or active low. "Reset" is ambiguous. "Synchronous reset, active high" is not.

Use parameter expressions for widths. For parameterized widths, use the parameter name rather than a number: `dataWidth` rather than `8`. This makes clear that the width is configurable.

Do not document clock and reset in the IO bundle. Chisel provides these implicitly. Document them in the port table but do not add them to your IO bundle.

6.2.5 Parameter Descriptions

The parameter descriptions live in the User Guide `params.tex` and document every parameter in your `<CoreName>Params` case class. The ADR system cross-references the parameter table

against your Scala source and flags discrepancies — missing parameters, mismatched default values, or inconsistent bounds all constitute conformance failures.

Document each parameter with its name, type, default value, permitted range, and a functional description:

```
\begin{tabular}{lllll}
  \hline
  \textbf{Parameter} & \textbf{Type} & \textbf{Default} &
  \textbf{Range} & \textbf{Description} \\
  \hline
  width & Int & 1 & 1--128 &
    Width of the data path in bits \\
  \hline
\end{tabular}
```

Write the parameter documentation at the same time as you write the `require` statements in the case class. The bounds in the table must exactly match the bounds enforced by `require` — the ADR system checks this. Writing them together eliminates the risk of drift.

6.2.6 Simulation

The simulation section lives in `sim.tex` and tells Core Users how to run the verification suite against your core. Include:

- The target frequency used in verification
- Instructions for running the full test suite
- A description of the test configurations and what each covers
- Instructions for interpreting test results
- Any known simulation limitations or requirements

The simulation instructions should be specific enough that a Core User can reproduce your verification results independently:

```
Run the complete verification suite using:

\begin{lstlisting}
make cov
```

This runs all directed tests across the following configurations:

- `config8` – 8-bit data width
- `config32` – 32-bit data width
- `config64` – 64-bit data width

6.2.7 Synthesis

The synthesis section lives in `syn.tex` and tells Core Users how to run synthesis and static timing analysis. This section shall include the target frequency — the CWS requires timing closure at the frequency stated here, so be accurate.

Include:

- The target clock frequency in MHz
- Instructions for running synthesis
- A description of the synthesis configurations and what each represents
- Area and timing results for each configuration
- The standard cell library used for synthesis
- Any synthesis constraints or recommendations

```
The target clock frequency is 200~MHz (5.0~ns period).

Run synthesis and static timing analysis using:

\begin{lstlisting}
make yosys
```

Synthesis is performed using Yosys with the Nangate 45nm open-source standard cell library. Three configurations are synthesized:

- `small_1` – 1-bit data width (minimum)
- `medium_64` – 64-bit data width
- `large_128` – 128-bit data width (maximum)

6.3 The CHANGELOG

The `CHANGELOG.md` file at the repository root records all changes to your core across every released version. It follows the Keep a Changelog convention:

https://keepachangelog.com

Each release entry shall include the version identifier, the release date, and a categorized list of changes:

```
## [1.0.0] - 2026-01-15
### Added
- Initial certified release
- Support for configurable data width (1 to 128 bits)
- Three synthesis configurations: 1-bit, 64-bit, 128-bit

## [0.1.0] - 2025-12-01
### Added
- Initial development version
```

The permitted change categories are: Added, Changed, Deprecated, Removed, Fixed, and Security. Use whichever categories apply to each release — not every category needs to appear in every release entry.

The version identifier in the most recent `CHANGELOG.md` entry shall match the version in the Git release tag and `build.sbt`. The ADR system checks this consistency automatically.

6.4 The README

The `README.md` file at the repository root is the first thing a Core User sees when they visit your core's repository. It should give them an immediate understanding of what the core does and how to get started.

The `README.md` shall contain at minimum:

- The core name and a brief functional description
- Your Organization ID and Team ID
- A link to the generated User Guide PDF
- Instructions for running `make all`

Keep the README concise. Its job is to orient the reader and point them to the User Guide for details — not to duplicate the User Guide content. A Core User who needs detailed port and parameter information should be directed to the User Guide, not required to scroll through a long README.

6.5 Scaladoc

Scaladoc comments in your Scala source files serve two purposes in chiselWare: they document your code for future maintainers, and they are parsed by the ADR system to verify consistency with the User Guide.

All public classes, methods, and parameters shall be documented with Scaladoc comments. The `<CoreName>Params` case class in particular requires careful Scaladoc documentation because the ADR system cross-references it against the User Guide parameter table.

Here is the required Scaladoc structure for the parameter case class:

```
/** Default parameter settings for <CoreName>
  *
  * @constructor
  *   Create a new <CoreName> parameter set
  * @param width
  *   Specifies the width of the data path in bits.
  *   Valid range: 1 to 128.
  * @author
  *   Your Name
  * @version
  *   1.0.0
  */
case class <CoreName>Params(width: Int = 1) {
  ...
}
```

The `@param` description for each parameter shall be consistent with the description in the User Guide parameter table. The ADR system checks this consistency — a mismatch between Scaladoc and the User Guide constitutes a conformance failure.

For the narrative style guidance on writing good Scaladoc comments, see Section 4.8 in Chapter 4.

6.6 License Documentation

The license documentation requirements depend on the license type of your core.

For **Apache 2.0** cores, `LICENSE.md` shall contain your copyright notice followed by the unmodified Apache 2.0 license text:

```
Copyright (c) <year> <Your Name or Organization>

Licensed under the Apache License, Version 2.0...
```

For **CW-GL** and **CW-CL** cores, `LICENSE.md` shall contain the SPDX identifier and contact details for license inquiries:

```
SPDX-License-Identifier: LicenseRef-chiselWare-GL

License inquiries:
  <Organization name>
  <Contact name or role>
  <Email address>
```

Every Scala source file shall begin with the SPDX copyright header on the first two lines:

```
// Copyright (c) <year> <Your Name or Organization>
// SPDX-License-Identifier: Apache-2.0
```

Replace `Apache-2.0` with `LicenseRef-chiselWare-GL` or `LicenseRef-chiselWare-CL` as appropriate for your core's license type.

6.7 Authorship and Provenance

chiselWare Cores are built primarily from original work. Occasionally you may incorporate a small amount of third-party source code — but only under strict conditions defined normatively in Chapter 14 of the CWS. This section explains how to document authorship and satisfy those provenance requirements in practice.

The two rules worth committing to memory before you copy any external code into your core: it must be licensed under **Apache 2.0** (no other license is permitted, however permissive), and code with no identifiable license is **prohibited** — absence of a license is not permission. When in doubt, write the code yourself.

6.7.1 The AUTHORSHIP.md File

Every chiselWare Core shall include an `AUTHORSHIP.md` file at the repository root. It serves two purposes: it records the people who contributed to the core, and it is the ledger of any third-party code incorporated under the CWS licensing rules. The Static Compliance Checks read this file when verifying provenance, so it is a required deliverable even for a core that is entirely original work.

The file has three sections. The first lists contributors.

The second records incorporated third-party code, or states plainly that there is none. The "None" statement is not a placeholder to be left blank — it is an affirmative declaration that the core contains no third-party code. The Static Compliance Checks expect either this declaration or a populated ledger.

The third relates the degree to which AI tools were used in the development of the core and an affirmation that developers have reviewed all generated code.

For a core written entirely from scratch — the common case — `AUTHORSHIP.md` is trivial:

```
# Authorship

## Contributors
- <Name> <email> -- original author(s)

## Incorporated Third-Party Code
None. All code in this Core is original work.

## Use of AI Tools
Check exactly one box.

- [X] No AI tools were used to create any part of this Core.
- [ ] AI tools provided minor assistance, such as autocomplete, small
      snippets, or refactoring suggestions reviewed and edited by hand.
- [ ] AI tools generated substantial portions of this Core, such as whole
      modules, testbenches, or documentation sections.

The contributors listed above have reviewed every part of this Core,
understand it, and take responsibility for it.
```

6.7.2 Recording Incorporated Code

When you do incorporate Apache 2.0 code, two things must happen: the source file itself must preserve the original attribution, and `AUTHORSHIP.md` must record the provenance.

In the source file, preserve the original copyright and license notice exactly — never remove, replace, or overwrite it. Add your own copyright line beneath it rather than altering the original, and add a provenance comment identifying where the code came from:

```
// SPDX-FileCopyrightText: OpenChiselIP Contributors
// SPDX-License-Identifier: Apache-2.0
//
// Portions of this file incorporate the baud-rate divisor logic from
// OpenChiselIP/uart-utils v1.2.0 (commit a1b2c3d), used under the
// Apache License, Version 2.0. Modified to parameterize the divisor
// width.
//

// SPDX-FileCopyrightText: Jane Developer
// SPDX-License-Identifier: Apache-2.0

package org.chiselware.cores.o00.t000.myuart
...
```

Then record the same incorporation in `AUTHORSHIP.md`:

```
# Authorship

## Contributors
- Jane Developer <jane@example.com> -- original author

## Incorporated Third-Party Code

### src/main/scala/.../BaudGen.scala
- Source: OpenChiselIP/uart-utils v1.2.0
- Origin commit: a1b2c3d
- License: Apache-2.0
- Original copyright: Copyright (c) 2024 OpenChiselIP Contributors
- Notes: Baud-rate divisor logic. Modified to parameterize the
  divisor width.
```

The two records serve different readers. The source-file header travels with the code wherever it goes and satisfies the Apache 2.0 attribution requirement directly. The `AUTHORSHIP.md` entry gives Core Users and reviewers a single place to see, at a glance, every piece of external code in the core and where it came from.

6.7.3 Upstream NOTICE Files

Apache 2.0 projects sometimes ship a `NOTICE` file containing required attribution text. If the source you incorporate carries a `NOTICE`, copy the relevant portions into your `AUTHORSHIP.md` under the corresponding entry. The Apache 2.0 license requires that `NOTICE` content be carried forward — dropping it is a license violation, not merely a documentation lapse.

6.7.4 Mixed-License Repositories

A single upstream repository is not always uniformly licensed. A project may be Apache 2.0 overall while vendoring a file under a different license, or the reverse — a GPL project containing one Apache 2.0 utility. The license that matters is the one on the **specific file** you incorporate, not the license the repository advertises at its root.

Before incorporating a file from a mixed-license repository, confirm the individual file's provenance:

- Check for a per-file SPDX identifier or copyright header — this governs, and overrides the repository-level license
- Check whether the file lives under a subdirectory with its own `LICENSE` file
- If the file carries no identifiable license of its own, treat it as prohibited — the repository-level Apache 2.0 license does not automatically extend to a file that declares something else, and a file with no notice at all is "all rights reserved"

Record the confirmed per-file license in your `AUTHORSHIP.md` entry, not the repository's advertised license, so the provenance ledger reflects what actually governs the incorporated code.

6.7.5 When in Doubt, Write It Yourself

The CWS makes incorporation the exception, not the default. If you cannot confidently identify a file's license, if the attribution chain is murky, or if confirming provenance is taking longer than the code would take to write, write an original implementation instead. Original authorship is always conformant and never requires a provenance investigation.

6.8 Checking Documentation Compliance

The ADR system performs extensive automated checks on your documentation. You do not need to wait for formal ADR submission to find documentation problems — the `make docs` target generates the documentation and produces a `doc.rpt` report that surfaces many issues locally:

```
make docs
```

Common documentation failures reported by ADR include:

- **DOC-003** — a port in the RTL is missing from `ports.tex`, or a port in `ports.tex` does not exist in the RTL
- **DOC-004** — a port width in `ports.tex` does not match the width in the RTL
- **DOC-005** — grammar or spelling issues in the `.tex` files
- **DOC-006** — a port description is too short or uninformative

Fix these issues before submitting for certification. Chapter 8 covers the full ADR submission process and how to interpret and respond to the complete compliance report.

When your documentation is complete and `make docs` produces a clean report, move on to Chapter 7 to run the full regression suite and prepare for certification submission.

Chapter 7

Running the Regression

By this point you have a working core, a verified test suite with adequate coverage, and complete documentation. The regression harness ties all of these together into a single automated workflow that verifies every aspect of your core in one command. This chapter explains what `make all` does, how to interpret its results, and how to prepare for certification submission.

7.1 The make all Sequence

`make all` is the single entry point for the complete chiselWare regression. It executes eight steps in a fixed order:

Step	Target	Description
1	`clean`	Remove all generated artifacts and reports from previous runs
2	`conformance`	Run tests to confirm template integrity
3	`lint`	Run `scalafmt` and `scalafix` checks
4	`publish`	Publish the core library locally via `publishLocal`
5	`cov`	Run all tests with Scoverage coverage enabled
6	`yosys`	Regenerate Verilog and run synthesis and static timing analysis
7	`docs`	Generate Scaladoc API documentation and the User Guide PDF
8	`ipf`	Generate IP Factory deliverables
9	`check`	Scan all reports and generate `error.rpt`

7.2 Running Individual Targets

During development you will frequently want to run individual steps rather than the full `make all` sequence:

```
make clean    # Remove all generated artifacts
make lint     # Check coding style only
make publish  # Publish core library locally
make test     # Run tests without coverage (faster)
make cov      # Run tests with coverage measurement
make verilog  # Generate Verilog only
```

```
make yosys    # Generate Verilog and run synthesis
make docs     # Generate Scaladoc and User Guide PDF
make ipf      # Generate IP Factory deliverables
make check    # Scan reports and update error.rpt
make list     # List all available targets
```

Use `make test` during active development for the fastest feedback loop — it skips coverage instrumentation which adds significant runtime. Use `make verilog` to quickly verify that your Chisel compiles to valid Verilog without running the full synthesis flow.

Note that `make publish` must be run before `make test` or `make cov` if you have just run `make clean` — the core library must be published locally before the tests can compile.

7.3 Interpreting Results

7.3.1 The Error Report

The authoritative pass/fail signal for every regression run is `error.rpt` at the repository root:

```
error.rpt
```

The rules are simple:

- File absent — regression failed
- File present and non-empty — regression failed; contents identify the cause
- File present and empty — regression passed

On a successful run the final output confirms:

ALL TESTS PASSED WITH NO ERRORS

On a failed run the output confirms:

TESTS COMPLETED WITH ERRORS

When the regression fails, start with `error.rpt` to identify which steps failed, then look at the corresponding report files for details:

Report	What to look for
`conformance.rpt`	Errors related to improperly customized templates
`ipf.rpt`	Errors related to IPF API tests
`lint.rpt`	File, line, and rule for each style violation
`docs/publish.rpt`	Errors and warnings from `publishLocal`
`docs/doc.rpt`	Scaladoc and User Guide generation errors
`generated/verilog.rpt`	Verilog generation errors
`generated/test.rpt`	Failed test names and assertion details
`generated/scalaCoverage/`	HTML coverage report showing uncovered lines and branches
`generated/synTestCases/*/`	Per-configuration Yosys logs and timing reports
`generated/synTestCases/timing_summary.rpt`	Timing closure status across all configurations
`generated/synTestCases/area_summary.rpt`	Area utilization across all configurations

7.3.2 Automatic Browser Launch

Several targets open generated files in the browser automatically when they complete:

- `make cov` — opens the Scoverage HTML coverage report
- `make docs` — opens the Scaladoc API documentation and the User Guide PDF

This is a deliberate reminder that these are living deliverables regenerated on every `make all` run. Review them regularly during development rather than waiting until you think you are done.

7.3.3 The GitHub Actions Workflow

The GitHub Actions workflow runs `make all` automatically on every pull request targeting `staging`. The workflow uses the same SSE Docker image and the same Makefile entrypoint as your local regression, so a passing local run should produce a passing CI run.

When a CI run fails, `error.rpt` is always uploaded to the workflow run as an artifact regardless of pass or fail status — download it and use it as your starting point for diagnosis exactly as you would for a local failure.

The most common cause of a CI failure that passes locally is a file that exists on your machine but was not committed to the repository. Run `git status` before pushing to ensure all required files are tracked.

7.4 Preparing for Certification Submission

Before opening a pull request to `staging` as a release candidate, work through this pre-submission checklist:

1. `make all` passes cleanly with an empty `error.rpt`
2. The version in `build.sbt`, the User Guide version history, and `CHANGELOG.md` all match and are set to the intended release version
3. AUTHORSHIP.md is present and accurately reflects all incorporated third-party code (or declares 'None')
4. The version is `1.0.0` or greater — pre-release versions cannot be certified
5. All simulation configurations pass in `generated/test.rpt`
6. Statement coverage is at or above 95% and branch coverage is at or above 90%
7. If formal verification is implemented, all formal properties pass for all `simConfigMap` configurations and results appear cleanly in `generated/test.rpt`
8. If formal verification is not implemented, then the following files have been deleted from the repository: `<CoreName>FormalTb.scala`, `<CoreName>FormalTest.scala`
9. All synthesis configurations achieve timing closure
10. The User Guide PDF looks correct — check all sections, tables, and figures
11. `ports.tex` matches the current RTL port list exactly
12. `params.tex` matches the current case class parameters exactly
13. All `require` constraints have corresponding `assertThrows` tests
14. All Scala source files have the correct SPDX copyright header
15. `LICENSE.md` contains the correct license for your core type
16. The `.ipf/` directory contains all required IP Factory deliverables
17. `README.md` is accurate and contains all required content
18. `CHANGELOG.md` has an entry for the release version

This checklist maps directly to the Annex A conformance checklist in the CWS. Working through it before submission will identify most conformance failures before the formal ADR run, saving time and iteration cycles.

When this checklist is complete, open a pull request from your feature branch to `staging` and let the CI regression confirm your local results. Chapter 8 covers the ADR submission process and the path to a certified chiselWare Core.

Chapter 8

Submitting for Certification

chiselWare Certification is the formal determination that your core meets every requirement of the chiselWare Standard. It is what authorizes you to use the chiselWare® name and logo in association with your core and makes your core eligible for listing in the chiselWare Core Registry and the IP Factory™ marketplace.

This chapter covers the certification submission process from the moment your local regression passes through to receiving your certification and publishing your core.

8.1 Before You Submit

Certification begins with a pull request from your feature branch to `staging`. Before opening that pull request, confirm that the pre-submission checklist in Chapter 7 Section 7.4 is complete. In particular:

- `make all` passes cleanly with an empty `error.rpt`
- The version is set to `1.0.0` or greater
- All three simulation and synthesis configurations pass
- Coverage thresholds are met
- The User Guide is complete and accurate

A submission that fails the CI regression on `staging` will not proceed to ADR. Fix all regression failures first.

8.2 The Certification Workflow

The certification workflow follows the branch promotion model established in Chapter 3:

Step 1 — Open a Pull Request to staging

Open a pull request from your feature branch to `staging` and label it as a release candidate in the PR description. The CI regression runs automatically. All checks must pass before proceeding.

Step 2 — Merge to staging

Once CI passes, merge the pull request to `staging`. The chiselWare Administrator is notified automatically that a release candidate is awaiting review.

Step 3 — ADR Run

The chiselWare Administrator initiates the Automated Design Review against your core on `staging`. The ADR system runs all conformance checks defined in Annex A of the CWS and produces a structured HTML compliance report. This process typically takes minutes for the automated checks, though the overall review timeline depends on the current certification queue.

Step 4 — Review the Compliance Report

The chiselWare Administrator shares the ADR compliance report with you via the IP Factory. Section 8.3 covers how to read and respond to the report.

Step 5 — Resolve Findings

If the ADR report contains failures, address each one on a new feature branch, re-run `make all` locally to confirm, and open a new pull request to `staging`. The ADR process repeats until all findings are resolved.

Step 6 — Promotion to main

Once the ADR report is clean, the chiselWare Administrator promotes your core from `staging` to `main` via a pull request and applies the release tag `v1.0.0`. This is the immutable marker that your core has passed the full certification process.

Step 7 — Registry and Marketplace Listing

The IP Factory updates the chiselWare Core Registry with your core's certification status and publishes the marketplace listing automatically. Your core is now a certified chiselWare Core and you are authorized to use the chiselWare logo in association with it.

8.3 The ADR Compliance Report

The ADR compliance report is an HTML document organized into six sections matching the six categories of ADR rules. Each rule shows a pass or fail status with a detailed explanation of any violations found.

Here is an example of the compliance report summary:

And here is an example of a detailed rule failure:

8.3.1 Reading the Report

Each rule failure in the report includes:

- The rule identifier (e.g. `SCALA-022`, `DOC-003`)



Figure 8.1: ADR compliance report summary showing pass/fail status for each of the six rule categories.

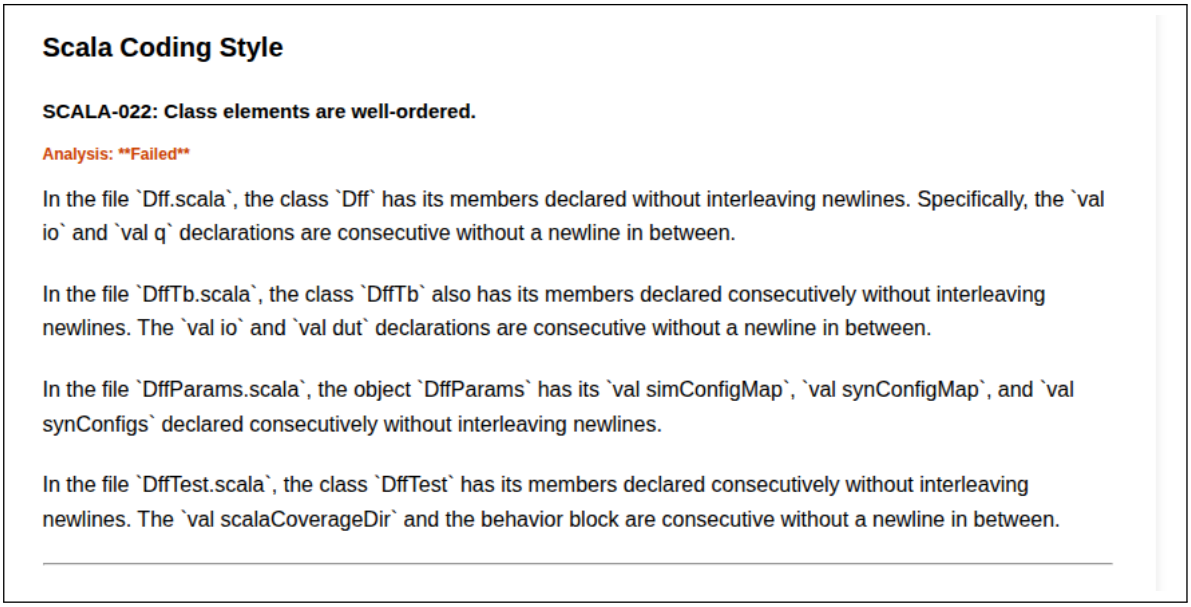


Figure 8.2: ADR compliance report detail showing a specific rule failure with explanation and guidance for resolution.

- A pass or fail status
- A detailed explanation of the violation
- In many cases, specific file names, line numbers, or field names where the violation was found

Use the rule identifier to look up the full rule description and resolution guidance in Annex A of the CWS. Annex A explains what each rule checks and provides context for understanding why the rule exists.

8.3.2 Verification Mechanism

The ADR system uses two types of checks as defined in CWS Section 3.4:

Static Compliance Checks — deterministic checks performed by `scalafmt` and `scalafix`. These produce definitive pass/fail results with no ambiguity. If your `make lint` passes locally, these checks will pass in ADR.

LLM-based checks — checks that require trained judgment to evaluate, such as documentation quality, narrative adequacy, and coding style consistency beyond what formatters can detect. These checks apply reasoning to evaluate conformance and may occasionally produce findings that require interpretation.

8.3.3 False Positives and Human Review

The ADR system may produce false positives — findings that are technically flagged but do not represent a genuine conformance failure. If you believe an ADR finding is incorrect, you may request a human review by the chiselWare Administrator.

To request human review, respond to the compliance report via the IP Factory with:

- The rule identifier of the disputed finding
- Your explanation of why you believe it is a false positive
- Any supporting evidence from your source code or documentation

The chiselWare Administrator will review the disputed finding and either confirm it as a genuine violation or issue a waiver. Waivers are recorded in the chiselWare Core Registry and are specific to the core version and rule for which they were granted. A waiver does not carry forward to future versions of your core — each new version is evaluated independently.

8.4 Resolving ADR Findings

Most ADR findings fall into predictable categories. The following guidance covers the most common ones.

8.4.1 Scala and Chisel Style Findings

Style findings that were not caught by `make lint` locally are rare but possible if the ADR system applies additional checks beyond `scalafmt` and `scalafix`. Fix these on a feature branch and confirm with `make lint` before resubmitting.

8.4.2 Documentation Consistency Findings

Documentation findings are the most common ADR failures for first submissions. The most frequent causes are:

- A port in `ports.tex` that does not match the RTL — update whichever is wrong and rerun `make docs` to confirm
- A parameter bound in `params.tex` that does not match the corresponding `require` constraint — update both together to ensure they stay in sync
- A parameter documented in the User Guide that is missing from `IpfParamsMap` — add the missing entry
- Port descriptions that are too short or uninformative — expand the description to provide meaningful functional context

8.4.3 Coverage Findings

Coverage findings mean your statement or branch coverage fell below the required thresholds in the ADR environment. This occasionally differs from local results due to environment differences. Open the Scoverage HTML report, identify the uncovered code, add tests to cover it, and resubmit.

8.4.4 Synthesis Findings

Synthesis findings typically mean timing closure was not achieved for one or more configurations. Review the `timing_summary.rpt` to identify which configuration and which path is failing. Common resolutions are adding pipeline registers to the critical path or adjusting the SDC constraints to match the actual achievable frequency, with a corresponding update to the User Guide synthesis section.

8.5 After Certification

Once your core is certified and listed in the chiselWare Core Registry, a few things to keep in mind:

The logo. You are now authorized to use the chiselWare logo in your core's documentation and marketing materials. The logo files are available from the chiselWare Administrator via the IP Factory. Use them in accordance with the chiselWare brand guidelines.

Ratings. Core Users who download your core through the IP Factory can rate it. Ratings are visible to all users browsing the marketplace. Good documentation, responsive maintenance, and a clean certification history all contribute to strong ratings.

Your Certification Version. Your core is certified against a specific version of the CWS. When the chiselWare Administrator releases a new version of the standard, your core will be regression-tested against it and any failures will be reported to you. Updating to the new standard version is optional — your existing certification remains valid indefinitely. However staying current keeps your core marked as certified against the latest standard, which is visible to Core Users in the marketplace.

New releases. Every new release of your core — Major, Minor, or Patch — requires recertification before the chiselWare logo may be used with that release. The recertification process is the same

as the initial certification: update your core, run `make all`, open a pull request to `staging`, and initiate ADR. See Chapter 9 for guidance on versioning and maintaining a certified core.

8.5.1 Citing chiselWare in Academic Work

If you are using chiselWare as part of academic research — whether developing certified cores for a published design, using chiselWare as infrastructure for a research project, or building on the chiselWare methodology — please cite chiselWare in your publications. Citations help demonstrate the impact of open-source EDA infrastructure in the research community and support the continued development of chiselWare.

The DevKit repository includes a `CITATION.cff` file and a `CITATION.md` file with ready-to-use citation formats including BibTeX. GitHub also displays a "Cite this repository" button in the sidebar of the DevKit repository for quick access.

`https://github.com/chiselWare/devkit`

If your institution requires a specific citation format not covered by `CITATION.md`, contact the chiselWare Administrator at `admin@chiselware.org` and we will be happy to assist.

Chapter 9

Maintaining Your Core

Certification is not the end of the journey — it is the beginning of a relationship with your Core Users. A certified chiselWare Core is a living product that will need updates to fix bugs, add features, respond to user feedback, and stay current with evolving versions of the chiselWare Standard. This chapter covers how to manage that ongoing responsibility well.

9.1 Versioning Your Releases

chiselWare uses Semantic Versioning (`x.y.z`) to communicate the nature of changes to Core Users. The version number is not just a label — it is a signal that tells users whether a new release affects their existing integration. Choosing the right version increment is one of the most important things you do as a Core Developer.

9.1.1 When to Increment Each Position

Patch (`z`) — increment for any non-functional change. Documentation updates, additional test cases, corrections to comments, improvements to the User Guide, and fixes to the regression harness are all Patch-level changes. The generated Verilog is identical before and after a Patch release. Core Users can take a Patch update with confidence that nothing in their integration will change.

Minor (`y`) — increment for any functional change to the RTL that results in different gates being generated during synthesis. Bug fixes that change circuit behavior, performance optimizations that alter the netlist, and changes to reset state are all Minor-level changes. Adding formal verification properties to an existing core without changing the RTL is a Patch-level change — the RTL is unchanged even though new files are added. When you

Major (`x`) — increment when you add a new feature or capability. A new operating mode, a new parameter, a new port, or a significant change to the core's interface are all Major-level changes. When you increment Major, reset both Minor and Patch to zero.

9.1.2 Practical Examples

- Fixed a typo in the User Guide: `1.0.0` → `1.0.1`
- Added formal verification properties to an existing core: `1.0.1` → `1.0.2`

- Fixed a bug in the state machine that caused incorrect output under a rare input condition: `1.0.2` → `1.1.0`
- Added support for a new high-speed operating mode controlled by a new parameter: `1.1.0` → `2.0.0`
- Added three more test configurations to improve coverage: `2.0.0` → `2.0.1`

When in doubt between Minor and Major, ask: does this change affect how Core Users instantiate or connect the core? If yes, it is Major. If the instantiation and connection are unchanged and only the internal behavior changes, it is Minor.

9.1.3 Updating Version Records

When you increment the version, update all three version records together — the ADR system checks their consistency and a mismatch is a conformance failure:

1. `build.sbt` — the `version` field
2. The User Guide version history table in `<CoreName>.tex`
3. `CHANGELOG.md` — add a new entry for the release

Do this as a single commit so the three records are always in sync in your Git history.

9.2 The Recertification Process

Every new release of a chiselWare Core requires recertification before the chiselWare logo may be used with that release. There are no exceptions — Major, Minor, and Patch releases all require recertification.

A core may be recertified against any CWS version that is greater than or equal to its original Certification Version. There is no requirement to upgrade to the latest CWS version in order to release a new version of your core. A core originally certified against CWS 1.0.0 may continue to be recertified against CWS 1.0.0 for all subsequent releases, even as newer versions of the standard are published. This ensures that bug fixes and maintenance releases are never blocked by a mandatory standards upgrade.

If you choose to upgrade your core to a newer CWS version, that upgrade applies to the new release going forward. You cannot downgrade a core's Certification Version once it has been certified against a newer standard.

The recertification process is identical to the initial certification:

1. Make your changes on a feature branch
2. Run `make all` locally and confirm it passes
3. Work through the pre-submission checklist in Section 7.4
4. Open a pull request to `staging`
5. Wait for CI to pass
6. The chiselWare Administrator initiates ADR against the target Certification Version

7. Resolve any findings
8. The chiselWare Administrator promotes to `main` and applies the new release tag

For Patch releases the ADR run is typically fast since only non-functional changes are involved and most rule categories will pass without findings. For Major and Minor releases a more thorough review is expected.

Note that the flexibility to recertify against a prior CWS version applies only to existing certified cores releasing new versions. New cores submitted for initial certification shall be certified against the current version of the standard at the time of submission. This ensures that every new core entering the chiselWare ecosystem meets the latest quality standards from the outset.

If you are adding formal verification for the first time in a new release, ensure both formal files (`<CoreName>FormalTb.scala` and `<CoreName>FormalTest.scala`) are present and contain real properties for your core before submitting for recertification. If you are removing formal verification from a release, ensure both files are deleted before submission.

9.3 Responding to User Issues

Core Users report issues with your core through the GitHub Issues tracker on your core's repository. The repository is accessible to Core Users who have downloaded your core, providing a transparent and community-visible support channel. Responding promptly and professionally to user issues is important for your core's ratings and reputation in the marketplace.

When a user reports an issue:

Reproduce it first. Before making any changes, reproduce the reported issue in your test suite. Add a directed test that demonstrates the failure. This gives you a regression test that confirms the fix and prevents the issue from recurring in a future release.

Determine the version impact. Is this a documentation clarification (Patch), a behavioral fix (Minor), or does it require an interface change (Major)? Communicate the version impact to the user in the GitHub Issue before releasing so they know what to expect.

Document it in Errata. If you cannot fix the issue immediately, document it in the `errata.tex` section of the User Guide with a workaround if one exists. Release a Patch version with the errata update so users are informed. An undocumented known issue is worse than a documented one.

Close the loop. After releasing a fix, update the GitHub Issue with the release version that contains the fix and close it. Users who receive responsive support leave better ratings in the marketplace.

9.4 Keeping Up with New CWS Versions

The chiselWare Standard evolves over time. When the chiselWare Administrator releases a new version of the CWS, your certified core will be regression-tested against the new version and any failures will be reported to you.

Updating to a new CWS version is optional — your existing certification remains valid indefinitely under the Certification Version it was granted. However there are good reasons to stay current:

- Core Users can see your Certification Version in the marketplace listing. A core certified against a current CWS version signals active maintenance.
- New CWS versions may include improvements to the ADR rule set that benefit from your core being compliant.
- If you plan to release new versions of your core, you will need to certify against the current CWS version at that time — starting the update process early is easier than doing it under pressure.

When a new CWS version is released, the chiselWare Administrator publishes a migration guide describing what changed and what updates are needed. Major CWS version changes will also include a new Dev Kit version with an updated template. Review the migration guide before starting any update work.

9.4.1 Updating the SSE

New CWS versions always include a new SSE version with updated tool version pins. Updating to a new CWS version requires updating your development environment to the new SSE.

For Docker users, pull the new image:

```
docker pull ghcr.io/chiselware/dev-full:<new-version>
```

For Azure VM users, the chiselWare Administrator will publish a new VM image in the Azure Marketplace. Update your VM to the new image before beginning development against the new CWS version.

After updating the SSE, run `make all` to identify any failures introduced by toolchain changes before making any code changes. This gives you a clean baseline from which to start the migration.

9.5 Deprecating and Retiring a Core

If you decide to stop maintaining a core, there are two options:

Deprecation — the core remains in the Registry and marketplace but is marked as deprecated. Core Users can still access it but are notified that it is no longer actively maintained. To deprecate a core, notify the chiselWare Administrator via the IP Factory. Update the `errata.tex` section of the User Guide to document the deprecation status and, if possible, recommend an alternative core.

Retirement — the core is removed from the active marketplace listing. It remains accessible to Core Users who have already downloaded it but does not appear in new searches. Retirement is a significant step — discuss it with the chiselWare Administrator before proceeding, particularly if the core has active users.

In either case, your core repository remains in the chiselWare GitHub organization and your certification history is preserved in the Registry. chiselWare does not delete certified cores.

9.6 Transferring Ownership

If you change employers, retire, or otherwise want to transfer responsibility for a core to another developer or organization, contact the chiselWare Administrator via the IP Factory. Ownership

transfers require both parties to have valid Developer IDs and executed Developer Agreements. The transfer is recorded in the chiselWare Core Registry and the repository access controls are updated accordingly.

If your Organization's chiselWare presence needs to be transferred — for example as part of a company acquisition — the chiselWare Administrator handles this at the Organization level. All cores associated with the Organization are transferred together.

Appendices

Appendix A

chiselWare Standard Reference

This appendix provides a summary reference to the normative requirements of the chiselWare Standard (CWS) organized for quick lookup during development. For the full normative text of each requirement, refer to the corresponding section of the CWS. In the event of any conflict between this appendix and the CWS, the CWS takes precedence.

Appendix B

Standard Software Environment Deployment Guide

The Standard Software Environment (SSE) is defined normatively in Annex C of the chiselWare Standard (CWS). That annex specifies the complete version-pinned toolchain against which all chiselWare Cores are developed and certified. This appendix provides practical deployment instructions for each of the three supported deployment methods.

For the authoritative toolchain specification including pinned tool versions, refer to CWS Annex C. In the event of any conflict between this appendix and CWS Annex C, the CWS takes precedence.

Appendix C

CI/CD Architecture

This appendix describes the chiselWare CI/CD infrastructure for Core Developers who want a deeper understanding of how the regression harness operates. The developer-facing aspects of the CI/CD workflow are covered in Chapters 3 and 7.

Appendix D

Scala and Chisel Resources

This appendix collects learning resources, reference documentation, and community links for Scala and Chisel development. Resources are organized by topic and annotated with a brief description to help you choose the most appropriate starting point for your needs.

D.1 Learning Scala

D.1.1 Official Documentation

Scala Documentation — The official Scala documentation site, including getting started guides, tour of Scala, and API reference.

https://docs.scala-lang.org

Scala Style Guide — The normative Scala coding style reference. chiselWare follows this guide for all Scala source. Read this before writing your first chiselWare Core.

https://docs.scala-lang.org/style/

Scala API Reference — The complete Scala standard library API documentation.

https://www.scala-lang.org/api/current/

D.1.2 Learning Materials

Scala Book — A comprehensive introduction to Scala covering all major language features with worked examples. Available free online.

https://docs.scala-lang.org/scala3/book/introduction.html

Scala Exercises — Interactive exercises for learning Scala in the browser. Covers standard library, functional programming, and more.

https://www.scala-exercises.org

Functional Programming in Scala — Chiancano and Bjarnason. The definitive book on functional programming in Scala. Recommended for developers who want to use Scala's functional programming features effectively in their chiselWare Cores.

D.1.3 Build Tools

sbt Documentation — The official sbt build tool documentation, including getting started, configuration reference, and plugin development.

https://www.scala-sbt.org/documentation.html

scalafmt Documentation — Configuration reference for the scalafmt code formatter used in the chiselWare SSE.

https://scalameta.org/scalafmt/docs/configuration.html

scalafix Documentation — Reference for the scalafix lint and rewrite tool used in the chiselWare SSE.

https://scalacenter.github.io/scalafix/

Scoverage — Documentation for the Scala code coverage tool used in chiselWare regression.

https://github.com/scoverage/scalac-scoverage-plugin

D.2 Learning Chisel

D.2.1 Official Documentation

Chisel Documentation — The official Chisel documentation site, including getting started, language reference, and migration guides.

https://www.chisel-lang.org/docs/

Chisel Developer's Style Guide — The normative Chisel coding style reference. chiselWare follows this guide with specific clarifications documented in CWS Section 10.

https://www.chisel-lang.org/docs/developers/style/

Chisel API Reference — The complete Chisel library API documentation.

https://www.chisel-lang.org/api/

D.2.2 Learning Materials

Chisel Bootcamp — An interactive Jupyter notebook-based introduction to Chisel. The recommended starting point for developers new to Chisel. Covers basic hardware constructs, parameterization, and functional circuit generation.

https://github.com/freechipsproject/chisel-bootcamp

Digital Design with Chisel — Schoeberl. A textbook introduction to digital design using Chisel, covering combinational and sequential logic, finite state machines, and pipelined designs. Available free online.

https://github.com/schoeberl/chisel-book

Chisel Users Group — Community discussion forum for Chisel users and developers. A good place to ask questions about Chisel language features and get help from experienced users.

https://groups.google.com/g/chisel-users

D.2.3 Verification

ChiselTest Documentation — Reference documentation for the ChiselTest simulation and testing framework used in chiselWare verification.

https://github.com/ucb-bar/chiseltest

ScalaTest Documentation — Reference for the ScalaTest testing framework that ChiselTest builds on. Covers test styles, matchers, and assertions.

https://www.scalatest.org/user_guide

D.3 Open Source EDA Tools

D.3.1 Simulation

Verilator — The open-source Verilog/SystemVerilog simulator used as the primary chiselWare simulation backend. Verilator compiles RTL to C++ for fast simulation.

https://www.veripool.org/verilator/

Icarus Verilog — An alternative open-source Verilog simulator available as a secondary backend in ChiselTest.

http://iverilog.icarus.com

GTKWave — An open-source waveform viewer for inspecting VCD and FST simulation output. Useful for debugging simulation failures.

https://gtkwave.sourceforge.net

D.3.2 Synthesis and Timing

Yosys — The open-source synthesis framework used in chiselWare synthesis regression. Supports Verilog and SystemVerilog input and produces gate-level netlists.

https://yosyshq.net/yosys/

OpenSTA — The open-source static timing analysis tool used in chiselWare timing closure verification.

https://github.com/The-OpenROAD-Project/OpenSTA

Nangate 45nm Open Cell Library — The open-source standard cell library used for chiselWare synthesis regression. Based on a fictional 45nm process, freely available with no licensing restrictions.

https://si2.org/open-cell-library/

D.3.3 Documentation

draw.io — The open-source diagramming tool used for block diagrams in chiselWare User Guides. Diagrams are stored as `.drawio` files and exported to PNG for inclusion in LaTeX documents.

https://www.drawio.com

WaveDrom — The open-source timing diagram tool used for waveform diagrams in chiselWare User Guides. Diagrams are defined as JSON and rendered to PNG.

https://wavedrom.com

D.4 chiselWare Community

chiselWare Website — The official chiselWare website, including the Core Registry, IP Factory marketplace, and developer registration.

https://chiselware.org

chiselWare GitHub Organization — The GitHub organization hosting all certified chiselWare Core repositories, the Dev Kit, the CI workflow framework, and other infrastructure.

https://github.com/chiselWare

chiselWare Standard (CWS) — The normative reference document defining all requirements for chiselWare Certification.

https://github.com/chiselWare/devkit

00-000-dff Template Repository — The certified D flip-flop template repository used as the starting point for all new chiselWare Core development.

https://github.com/chiselWare/00-000-dff

Appendix E

Glossary

This glossary defines the key terms used throughout the chiselWare Developers Guide. Terms are defined from the perspective of a Core Developer working within the chiselWare ecosystem. For the normative definitions of these terms, refer to Section 3 of the chiselWare Standard (CWS).

Automated Design Review (ADR)
The automated conformance checking system operated by the chiselWare Administrator that verifies submitted cores against the requirements of the CWS. The ADR system combines deterministic static analysis tools with LLM-based analysis to evaluate conformance across coding style, documentation quality, synthesis deliverables, and more. ADR produces a structured HTML compliance report for each submission. See Chapter 8.

Certification Version
The version of the chiselWare Standard against which a chiselWare Core was certified. A core retains its certification under its Certification Version indefinitely. New releases of an existing core may be recertified against the same Certification Version or any later version. New cores must be certified against the current version of the standard at the time of submission.

chiselWare Administrator
The organizational role responsible for maintaining the CWS, operating chiselware.org and the IP Factory, administering the developer registration process, managing the chiselWare Core Registry, and authorizing use of the chiselWare trademark and logo.

chiselWare Certification
The formal determination by the chiselWare Administrator that a core satisfies all requirements of the CWS under a specific Certification Version. Certification is binary — a core is either certified or it is not. Certification authorizes use of the chiselWare name and logo in association with the certified core version.

chiselWare Core
An IP core implemented in Chisel that has achieved chiselWare Certification. Only chiselWare Cores may bear the chiselWare logo and be represented as chiselWare in marketing materials, documentation, or product listings. Contrast with Chisel Core.

chiselWare Core Registry

The authoritative database maintained by the chiselWare Administrator recording all chiselWare Cores, their certification status, their Certification Version, their license type, and their associated Organization and Team IDs. The Registry is maintained automatically by the IP Factory and is accessible at `https://chiselware.org`.

chiselWare Developer Agreement
The legal agreement executed between a Core Developer and the chiselWare Administrator at the time of registration. It establishes the terms under which cores may be submitted, certified, and distributed within the chiselWare ecosystem, including IP ownership warranties and platform fee terms.

chiselWare Standard (CWS)
The normative document defining all requirements for chiselWare Certification. The CWS specifies requirements for directory structure, versioning, parameterization, coding style, verification, documentation, and regression testing. This guide is the practical companion to the CWS — in the event of any conflict between the two, the CWS takes precedence.

Chisel Core
An IP core implemented in Chisel that has not achieved chiselWare Certification. A Chisel Core may conform to some or all requirements of the CWS but shall not be represented as a chiselWare Core and shall not bear the chiselWare logo or trademark. Contrast with chiselWare Core.

Core Developer
An individual who has completed registration at chiselware.org, received a Developer ID, and executed a chiselWare Developer Agreement. Only registered Core Developers may submit cores for chiselWare Certification. You are a Core Developer.

Core User
The individual or organization that integrates a chiselWare Core into a semiconductor design. Core Users interact with your core through the IP Factory marketplace and through the documentation you provide. The Core User's experience is the primary motivation behind chiselWare's quality requirements.

Developer ID
A unique identifier assigned by the chiselWare Administrator upon successful registration at chiselware.org. The Developer ID is associated with your executed chiselWare Developer Agreement and is a prerequisite for core submission and certification.

error.rpt
The authoritative pass/fail signal for every chiselWare regression run. Generated by the `make check` target at the repository root. An empty file indicates a passing regression. A non-empty file contains the aggregated errors from all regression steps. See Chapter 7.

Independent Organization (oIN)
A reserved Organization provided by the chiselWare Administrator for Core Developers who have no formal institutional affiliation, or who are developing cores independently of their employer's chiselWare presence. Cores developed under `oIN` carry no implied affiliation with any other Organization. A developer may belong to both the Independent Organization and one or more

named Organizations simultaneously.

IP Factory
The operational hub of the chiselWare ecosystem. The IP Factory handles Core Developer and Core User registration via LinkedIn authentication, repository provisioning and management, access control for paid cores, license agreement execution, commercial transactions via Stripe integration, and the chiselWare Core marketplace. The IP Factory also maintains the chiselWare Core Registry automatically. See Chapter 1.

IpfParamsMap
A Scala object in `<CoreName>Params.scala` that defines the parameter metadata used by the IP Factory to present your core's configurable parameters in the marketplace. Every parameter in your case class must have a corresponding entry. See Section 4.4.

Organization
A named entity registered at chiselware.org representing a company, academic institution, or independent developer group. An Organization is assigned a unique two-character Organization ID by the chiselWare Administrator. All chiselWare Cores are associated with an Organization and a Team.

Organization ID
A two-character alphanumeric identifier assigned by the chiselWare Administrator to a registered Organization. The Organization ID appears in the chiselWare package namespace for all cores developed under that Organization, in the form `o<OrgID>`.

Semantic Versioning (SemVer)
The versioning scheme used by all chiselWare Cores, of the form `x.y.z` where `x` is the Major version, `y` is the Minor version, and `z` is the Patch version. Major increments signal new features, Minor increments signal functional RTL changes, and Patch increments signal non-functional changes. See Chapter 9.

simConfigMap
A `LinkedHashMap` in the `<CoreName>Params` companion object that defines the named parameter configurations exercised by the simulation test suite. The test suite iterates over every entry in `simConfigMap` automatically, producing composite code coverage across all configurations. At minimum three configurations are required: default, minimum, and maximum. See Section 4.2.

Standard Software Environment (SSE)
The version-pinned toolchain against which all chiselWare Cores are developed and certified. The SSE includes Chisel, Scala, sbt, simulation tools, synthesis tools, documentation tools, and code quality tools at specific pinned versions chosen to work together without conflict. The SSE is distributed as a Docker image, as an Azure VM, and as a published toolchain specification in Annex D of the CWS. See Appendix B.

Static Compliance Check (SCC)
A conformance check performed by `scalafmt` or `scalafix` rather than by the ADR rule engine. Static Compliance Checks produce deterministic pass/fail results. If your `make lint` passes locally, Static Compliance Checks will pass in ADR.

synConfigMap
A `LinkedHashMap` in the `<CoreName>Params` companion object that defines the named parameter configurations synthesized by the Yosys synthesis flow. Map keys use snake_case by convention since they appear in Verilog-facing synthesis scripts. At minimum three configurations are required: default, minimum, and maximum. See Section 4.2.

Team
A named group of Core Developers operating within a registered Organization. A Team is assigned a unique three-character Team ID by the chiselWare Administrator. All chiselWare Cores are associated with a Team within an Organization.

Team ID
A three-character alphanumeric identifier assigned by the chiselWare Administrator to a registered Team. The Team ID appears in the chiselWare package namespace for all cores developed under that Team, in the form `t<TeamID>`.

00-000-dff
The certified D flip-flop template repository provided by the chiselWare Administrator as the mandatory starting point for all new chiselWare Core development. It provides a complete, fully compliant chiselWare Core that developers customize for their own design. Available at https://github.com/chiselWare/00-000-dff. See Chapter 3.

Waiver
A formal determination by the chiselWare Administrator that a specific ADR finding is a false positive and does not constitute a genuine conformance failure. Waivers are recorded in the chiselWare Core Registry and are specific to the core version and ADR rule for which they were granted. A waiver does not carry forward to future versions of the same core. See Chapter 8.